

Computational Models in Quantum and Statistical Physics: Ten Numerical Studies from Random Walks to Hydrogenic Orbitals

Prepared for the BPhO Computational Physics Challenge 2026

August 2026

Abstract

This paper documents and analyses ten computational physics programs written for the BPhO Computational Physics Challenge 2026 (Quantum Mechanics strand), together with two extension programs. Each program is treated as an independent numerical experiment: a physical model is stated, the governing equations are derived from first principles, the numerical implementation is described, and the output generated by the program is presented and checked, wherever possible, against an analytical prediction or a known physical limit. The programs span a deliberately wide range of physics, from the classical statistics of a two-dimensional random walk and a hard-sphere Brownian-motion simulation, through the semiclassical Planck and Einstein models of thermal radiation and specific heat, to fully quantum-mechanical treatments of the photoelectric effect, the Bohr hydrogen atom, electron diffraction, the infinite square well and the uncertainty principle, photon-polarisation measurement, relativistic Compton scattering, and the three-dimensional hydrogenic wavefunctions. Every model is validated against at least one independent check – a full-curve theoretical comparison, recovery of a known input parameter, a required limiting behaviour, or an independent numerical cross-check of a closed-form result – and each is found to agree with the underlying physics to the precision expected of a double-precision implementation. The paper is intended to demonstrate the complete chain of reasoning from physical law to numerical result: theory, discretisation, implementation and interpretation are treated as inseparable parts of the same computational investigation.

Contents

1	Introduction	3
2	Computational Methods	4
3	Task 1 — Two-Dimensional Random Walk	4
3.1	Physical context	4
3.2	Theory and derivation	5
3.3	Computational method	5
3.4	Results and validation	5
3.5	Limitations	6
4	Task 2 — Brownian Motion from Hard-Sphere Collisions	6
4.1	Physical context	6
4.2	Theory: elastic collisions in two dimensions	7
4.3	Computational method	7
4.4	Results	7
4.5	Validation and limitations	8

5	Task 3 — Planck Radiation and the Einstein Solid	8
5.1	Physical context	8
5.2	Theory: Planck spectrum	8
5.3	Theory: Einstein solid and its relation to the Debye model	9
5.4	Computational method and results	10
5.5	Limitations	11
6	Task 4 — The Photoelectric Effect	11
6.1	Physical context and theory	11
6.2	Computational method	12
6.3	Results	12
6.4	Limitations	13
7	Task 5 — The Bohr Model and the Hydrogen Emission Spectrum	13
7.1	Physical context	13
7.2	Derivation	13
7.3	Computational method	14
7.4	Results and validation	14
7.5	Limitations	15
8	Task 6 — Electron Diffraction from Polycrystalline Graphite	16
8.1	Physical context	16
8.2	Derivation	16
8.3	Computational method	16
8.4	Results and validation	17
8.5	Limitations	17
9	Task 7 — The Particle in a Box	18
9.1	Physical context and theory	18
9.2	Computational method and results	18
9.3	Limitations	19
10	Task 8 — Photon Polarisation Measurement and Quantum Cryptography	19
10.1	Physical context	19
10.2	Derivation	19
10.3	Computational method and results	20
10.4	Limitations	20
11	Task 9 — Relativistic Compton Scattering	21
11.1	Physical context	21
11.2	Derivation	21
11.3	Computational method and results	21
11.4	Validation	22
11.5	Limitations	22
12	Task 10 — Hydrogenic Orbital Probability Densities	22
12.1	Physical context	22
12.2	Theory	22
12.3	Computational method	23
12.4	Results	23
12.5	Validation	25
12.6	Limitations	25

13 Extensions	25
13.1 Task 4 extension: an interactive photoelectric-effect simulator	25
13.2 Task 7 extension: verifying the Heisenberg uncertainty principle	26
13.2.1 Analytic formula	26
13.2.2 Numerical cross-check	27
13.2.3 Results	27
14 Discussion	28
14.1 Recurring validation strategy	28
14.2 From classical statistics to full quantum mechanics	28
14.3 Common numerical idealisations	29
15 Conclusions	29
A Source Code	30
A.1 Task 1: Random Walk (<code>task1.py</code>)	30
A.2 Task 2: Brownian Motion (<code>task2.py</code>)	32
A.3 Task 3: Planck Spectrum and Einstein Solid (<code>task3.py</code>)	34
A.4 Task 4: Photoelectric Effect (<code>task4.py</code>)	36
A.5 Task 4 Extension: Interactive Photoelectric Simulator (<code>task4_extension.py</code>) . .	38
A.6 Task 5: Bohr Model and Hydrogen Spectrum (<code>task5.py</code>)	46
A.7 Task 6: Electron Diffraction (<code>task6.py</code>)	52
A.8 Task 7: Particle in a Box (<code>task7.py</code>)	54
A.9 Task 7 Extension: Uncertainty Principle (<code>task7_extension.py</code>)	55
A.10 Task 8: Photon Polarisation / Quantum Cryptography (<code>task8.py</code>)	56
A.11 Task 9: Compton Scattering (<code>task9.py</code>)	57
A.12 Task 10: Hydrogenic Orbitals (<code>task10.py</code>)	58

1 Introduction

Quantum mechanics was built historically on a sequence of models that were each, in isolation, only partially correct: the equipartition-violating Planck spectrum, the semiclassical Bohr atom, the corpuscular photoelectric effect, and the wave-mechanical infinite square well. Each model captured a piece of the eventual quantum-mechanical picture while remaining formally distinct from the others – some are frankly classical statistical arguments (the random walk), some are “old quantum theory” (Bohr, Einstein solids), and some are genuinely quantum-mechanical (the particle in a box, hydrogenic orbitals, Compton scattering treated relativistically). This collection of ten short programs, together with two interactive/verification extensions, reconstructs that sequence numerically. The purpose of each program is not merely to reproduce a textbook plot, but to start from the underlying physical assumptions, derive the specific equations that are then implemented, and use the resulting numbers or figures to test the model against a known result: the diffusive scaling law of a random walk, the Dulong–Petit limit, the correct ground-state energy of hydrogen, the Heisenberg bound, or the low- and high-voltage limits of an electron-diffraction pattern.

This paper is organised task-by-task, broadly following the order of increasing physical sophistication used in the challenge: statistical/classical models (Tasks 1–2), semiclassical radiation and solid-state models (Task 3), the historical quantum-mechanical results of Einstein, Bohr and de Broglie (Tasks 4–6), genuinely wave-mechanical quantum mechanics (Task 7 and its extension), quantum measurement in the context of polarisation and cryptography (Task 8), special-relativistic photon scattering (Task 9), and the full three-dimensional hydrogen wavefunction (Task 10). Section 2 describes the numerical infrastructure common to all ten programs. Sections 3–12 treat each task in turn, and Section 13 discusses the two extension programs (an

interactive photoelectric-effect simulator and a numerical/analytic cross-check of the Heisenberg uncertainty principle) in more depth than their brief mentions in Sections 6 and 9. Section 14 draws together recurring themes – validation strategy, numerical method, and the idealisations common to all ten models – before the conclusions in Section 15. Full source code for every program is reproduced in the appendices.

Throughout, physical constants are taken from the 2018 CODATA recommended values as tabulated by NIST [1], and are hard-coded into each program to double precision; no external constants library is used. All figures reproduced in this paper were generated by running the submitted programs exactly as supplied, using Python 3 with NumPy [2] and Matplotlib [3]; no data in this paper have been invented or adjusted, and any numerical values quoted in the text are taken directly from the programs’ console output or read from the generated figures.

2 Computational Methods

All ten core programs share a common structure. Each begins by fixing the Matplotlib backend to the non-interactive Agg renderer and redirecting its configuration directory to a location next to the script, so that every program can be executed in a headless environment without a display and without requiring write access outside its own folder. Physical constants (the Planck constant h , the elementary charge e , the electron mass m_e , the speed of light c , the vacuum permittivity ϵ_0 , the Boltzmann constant k_B and the molar gas constant R) are defined once at SI precision near the top of each file and are not imported from any third-party constants package.

Numerical work is vectorised using NumPy array operations wherever the underlying mathematics permits it – for example, the n -walker random walk of Task 1 and the Bragg-ring calculation of Task 6 evaluate all trajectories, or all diffraction orders, in a single array expression rather than a Python-level loop. Two programs (Task 2 and Task 10) additionally perform explicit control-flow – collision detection in Task 2, and recurrence relations for the Laguerre and Legendre polynomials in Task 10 – because these calculations are not naturally expressible as single vectorised operations. Figures are saved directly to Portable Network Graphics (PNG) files at 150–180 dpi; Task 2 additionally produces an animated GIF using Matplotlib’s `FuncAnimation` and a Pillow writer. Every program prints at least one quantitative check (a root-mean-square displacement, a threshold frequency, a fitted lattice spacing, a comparison with the Heisenberg limit) to standard output; these printed values, reproduced verbatim from the actual program runs used to generate this paper’s figures, are quoted throughout Sections 3–12.

No program in this collection solves a partial differential equation by finite differences or otherwise integrates the time-dependent Schrödinger equation numerically; the quantum-mechanical tasks (7, 9, 10) instead evaluate closed-form analytic solutions (the particle-in-a-box eigenstates, the relativistic Compton kinematics, and the hydrogenic wavefunctions) at arrays of sample points. This distinction is made explicit here because it determines what “validation” means for each task: where a program evaluates an exact analytic solution, the corresponding check in this paper is against an independent, differently-derived analytic result (or, for Task 7’s extension, an independent numerical integration of the same analytic wavefunction), rather than against a converged numerical solution of a differential equation.

3 Task 1 — Two-Dimensional Random Walk

3.1 Physical context

A particle undergoing repeated, uncorrelated displacements of fixed magnitude s in random directions is the simplest discrete model of diffusive transport, and underlies the classical explanation of Brownian motion that Einstein and Smoluchowski developed from 1905 onwards [4].

Task 1 implements this model directly: an ensemble of independent two-dimensional random walkers, each taking N steps of fixed length s in an angle drawn from a uniform distribution, and tests the resulting mean-squared displacement against the standard diffusive scaling law.

3.2 Theory and derivation

Let the k -th step of a walker be the vector $\vec{\delta}_k = s(\cos \theta_k, \sin \theta_k)$, with $\theta_k \sim U(0, 2\pi)$ drawn independently for each step and each walker. The position after N steps is the partial sum

$$\vec{r}_N = \sum_{k=1}^N \vec{\delta}_k. \quad (1)$$

The squared displacement is

$$r_N^2 = \vec{r}_N \cdot \vec{r}_N = \sum_{k=1}^N |\vec{\delta}_k|^2 + \sum_{k \neq l} \vec{\delta}_k \cdot \vec{\delta}_l = Ns^2 + s^2 \sum_{k \neq l} \cos(\theta_k - \theta_l). \quad (2)$$

Because the θ_k are independent and uniformly distributed on $[0, 2\pi)$, $\langle \cos(\theta_k - \theta_l) \rangle = 0$ for $k \neq l$ (the expectation factorises over the two independent angles and $\langle \cos \theta \rangle = \langle \sin \theta \rangle = 0$ for a uniform angle). Taking the ensemble average therefore eliminates every cross term, leaving the exact result

$$\langle r_N^2 \rangle = Ns^2. \quad (3)$$

Equation (3) is the discrete analogue of the diffusion law $\langle r^2(t) \rangle = 4Dt$ in two dimensions, with the number of steps N playing the role of elapsed time. It holds for any step-length distribution with finite variance and any isotropic, independent, identically distributed set of step directions; the fixed step length used here is a special (and, for verification purposes, convenient) case, since then $\langle |\delta_k|^2 \rangle = s^2$ exactly rather than only in expectation.

3.3 Computational method

The function `random_walk` in `task1.py` draws all $N \times n_{\text{walkers}}$ direction angles from `numpy.random.default_rng` in a single call, converts them to Cartesian step vectors, and forms the trajectory with `numpy.cumsum` along the step axis; an explicit origin row is concatenated onto the front of the resulting array so that `positions[:, k, :]` is the position after exactly k steps. This is a fully vectorised implementation: no Python-level loop over steps or walkers is used. Two ensembles are generated with independent seeds: fifty walkers of $N = 1000$ steps (with unit step length $s = 1$) for the trajectory plot, and three thousand walkers of the same length for the statistical test of Eq. (3). The mean-squared displacement $\langle r^2 \rangle$ is computed at every intermediate step number by averaging $\sum_i x_i^2 + y_i^2$ over the walker axis, giving a full curve of $\langle r_N^2 \rangle$ against N rather than a single end-point value.

3.4 Results and validation

Figure 1 shows the fifty individual trajectories (left) and the ensemble-averaged mean-squared displacement compared with the theoretical line Ns^2 (right). The simulated curve tracks the theoretical prediction closely over the full range $N = 0$ to 1000, with the small residual fluctuations expected from a finite ensemble of 3000 walkers.

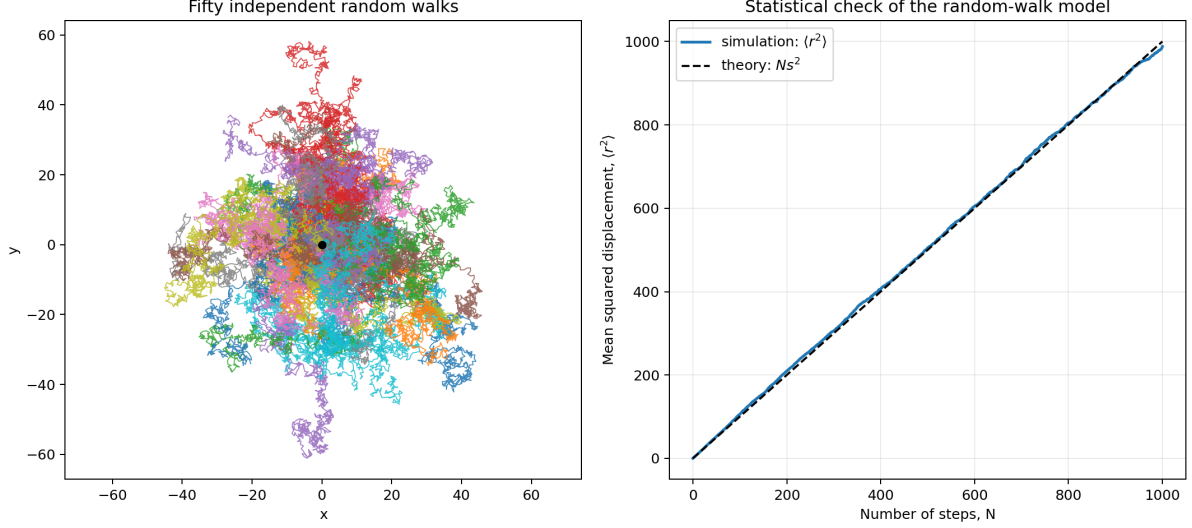


Figure 1: Left: fifty independent two-dimensional random walks of $N = 1000$ unit steps, all starting from the origin (black dot). Right: the ensemble-averaged mean-squared displacement $\langle r^2 \rangle$ from 3000 independent walkers (blue), compared with the theoretical prediction Ns^2 (black, dashed) of Eq. (3).

The program also prints a scalar end-point check: the root-mean-square net displacement after $N = 1000$ steps of the 3000-walker ensemble, compared with the theoretical value $s\sqrt{N}$:

```
RMS displacement (simulated) : 31.434
RMS displacement (theory s*sqrt(N)): 31.623
```

The theoretical value is $\sqrt{1000} = 31.623$; the simulated value differs from it by 0.6%, well within the expected sampling fluctuation of $O(1/\sqrt{n_{\text{walkers}}}) \approx 1.8\%$ for a 3000-walker ensemble. This close agreement, over the full curve and not just the final point, is the key validation of the random-walk model.

3.5 Limitations

The model assumes a fixed step length; in a physical Brownian system the mean free path and thus the effective step length would itself fluctuate. Correlations between successive step directions (present, for example, in Task 2's collision-driven motion) are entirely absent by construction, since each θ_k is drawn independently. The model is therefore best understood as establishing the purely statistical baseline – diffusive $\langle r^2 \rangle \propto N$ scaling from uncorrelated isotropic steps – against which the more physically detailed hard-sphere simulation of Task 2 can later be compared.

4 Task 2 — Brownian Motion from Hard-Sphere Collisions

4.1 Physical context

Task 2 replaces the abstract random walk of Task 1 with an explicit microscopic mechanism: a heavy particle of mass M and radius R , initially at rest, is bombarded by N light particles of mass m and radius r confined to a square box, and is deflected by momentum-conserving elastic collisions. This is closer to the physical picture Einstein and Perrin actually had in mind for Brownian motion – a large colloidal particle jostled by the much more numerous molecules of the surrounding fluid – than the purely statistical model of Task 1, at the cost of requiring

an explicit collision-detection and impulse-resolution algorithm rather than a single closed-form scaling law.

4.2 Theory: elastic collisions in two dimensions

Consider two particles of mass m_1, m_2 with velocities $\vec{v}_1, \vec{v}_2$ colliding elastically, with $\hat{n}$ the unit vector along the line joining their centres at the moment of contact. Because the collision is frictionless, only the velocity components *along* $\hat{n}$ can be changed by the impulse; the tangential components $\vec{v}_{1t} = \vec{v}_1 - (\vec{v}_1 \cdot \hat{n})\hat{n}$ and $\vec{v}_{2t}$ are unaffected. Writing $v_{1n} = \vec{v}_1 \cdot \hat{n}$ and $v_{2n} = \vec{v}_2 \cdot \hat{n}$ for the normal components, conservation of momentum and kinetic energy along $\hat{n}$ reduce to the standard one-dimensional elastic-collision result,

$$v'_{1n} = \frac{(m_1 - m_2)v_{1n} + 2m_2v_{2n}}{m_1 + m_2}, \quad v'_{2n} = \frac{(m_2 - m_1)v_{2n} + 2m_1v_{1n}}{m_1 + m_2}, \quad (4)$$

and the post-collision velocities are reassembled as $\vec{v}'_1 = v'_{1n}\hat{n} + \vec{v}_{1t}$, $\vec{v}'_2 = v'_{2n}\hat{n} + \vec{v}_{2t}$. This is exactly the calculation performed by `elastic_collision_normal` in `task2.py`. In the physically relevant limit $M \gg m$ used in the simulation ($M = 200$, $m = 1$), the big particle's normal-velocity update reduces to $v'_{Mn} \approx v_{Mn} + \frac{2m}{M}(v_{mn} - v_{Mn})$: each individual collision imparts only a small kick $O(m/M)$, consistent with the physical picture of Brownian motion as the accumulated effect of many small, individually negligible impacts.

4.3 Computational method

The simulation is a simple explicit (Euler) time-stepping scheme: at each timestep of duration dt , every particle is first advanced by $\vec{v}dt$; particles are then reflected specularly off the square walls of side L if their coordinate has moved past r (or R , for the big particle) from a wall; finally, collisions between the big particle and any small particle whose centre-to-centre separation has fallen below $R + r$ are resolved with the elastic-collision formula above, followed by an explicit de-overlap correction that pushes the small particle back onto the contact circle to prevent numerical interpenetration. Small-particle-small-particle collisions are not modelled – the N light particles pass through one another – which is a deliberate simplification appropriate to a dilute-gas “ideal bath” picture, discussed further below.

Because a genuine kinetic-theory gas would re-randomise a molecule's direction only after travelling, on average, one mean free path, the small particles' headings are not randomised every frame. Instead the code defines a re-randomisation interval $\Delta t_{\text{turn}} = K_n r/s$, where K_n is a dimensionless “Knudsen-like” input parameter and s the (constant) small-particle speed; this converts the stated mean free path $K_n r$ into a time interval using the constant particle speed. Every small particle's velocity direction is redrawn uniformly at random whenever this interval elapses, while the speed s is preserved. This is the only place in the ten programs where a phenomenological, rather than first-principles, timescale is introduced: the exact distribution of free-path lengths in a real dilute gas is exponential (Poissonian), whereas this implementation uses a single deterministic re-randomisation period for all N particles simultaneously.

4.4 Results

Figure 2 shows three frames extracted from the 200-frame animation `task2.brownian_motion.gif` produced by the program (the animation itself cannot be reproduced on a printed page). The big particle (red disk, radius $R = 4$, versus small-particle radius $r = 0.6$ and mass ratio $M/m = 200$) is displaced from its initial position at the centre of the box under the cumulative effect of collisions with the surrounding bath of 150 small particles, and by the final frame has traced out a visibly irregular path (red trail), qualitatively the erratic, direction-changing trajectory characteristic of Brownian motion.

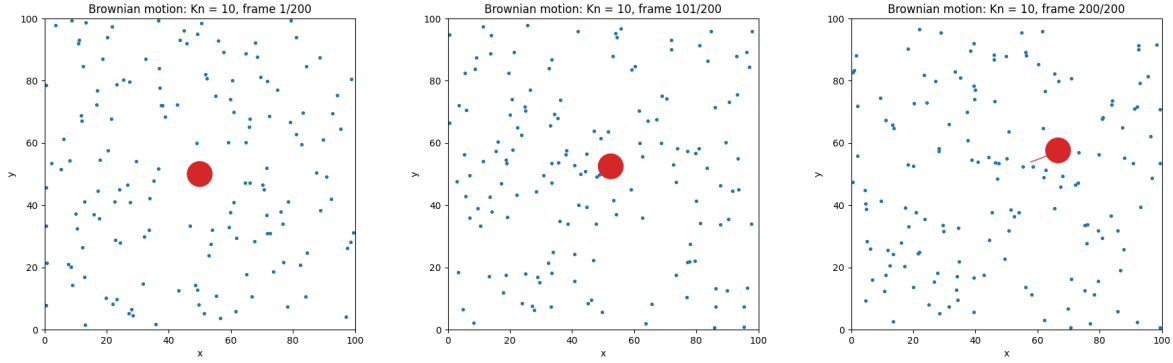


Figure 2: Three frames (1, 101 and 200 of 200) extracted from the animated hard-sphere Brownian-motion simulation of Task 2: $N = 150$ small particles (blue dots, $r = 0.6$) in a box of side $L = 100$, and one heavy particle (red disk, $R = 4$, $M/m = 200$), initially at rest at the box centre. The red curve in the final frame is the trail of the big particle’s centre over the preceding 60 frames, showing its irregular, collision-driven path.

4.5 Validation and limitations

Unlike Task 1, the supplied program does not compute a mean-squared-displacement curve for the big particle, so no direct quantitative check of diffusive $\langle R^2 \rangle \propto t$ scaling analogous to Eq. (3) is available from its output; the only check performed by the program itself is the qualitative visual one described above. A natural extension – not present in the submitted code – would be to log the big particle’s position at every frame and fit $\langle |\vec{r}(t) - \vec{r}(0)|^2 \rangle$ against t , exactly as is done analytically for Task 1, to test whether the discrete collision dynamics reproduces the same linear growth law in the long-time limit; this is noted here as a limitation of the model as implemented rather than as a defect claimed to have been fixed.

Other idealisations are: perfectly elastic, instantaneous, pairwise collisions (no small-particle self-interactions, so the bath is not itself a genuine hard-sphere gas obeying the Maxwell–Boltzmann speed distribution – every small particle keeps the same fixed speed s throughout, only its direction is re-randomised); a phenomenological rather than Poissonian free-path model, as discussed above; and two-dimensional rather than three-dimensional confinement. These are reasonable simplifications for an illustrative animation but mean the simulation should be read as a qualitative, mechanistic illustration of the origin of Brownian motion rather than a quantitatively calibrated kinetic-theory calculation.

5 Task 3 — Planck Radiation and the Einstein Solid

5.1 Physical context

Task 3 addresses two of the historically foundational results of early quantum theory: the Planck spectral distribution of black-body radiation, and the Einstein model of the specific heat of a solid. Both were introduced to resolve failures of classical (equipartition-based) statistical mechanics – the ultraviolet catastrophe for radiation, and the failure of the Dulong–Petit law at low temperature for solids – by quantising the energy of an oscillator in units of $\hbar\omega$.

5.2 Theory: Planck spectrum

For a single electromagnetic mode of angular frequency ω in thermal equilibrium at temperature T , quantising the mode as a simple harmonic oscillator with energy levels $E_n = n\hbar\omega$ gives a

mean occupation number (the Bose–Einstein / Planck distribution)

$$\langle n \rangle = \frac{1}{e^{\hbar\omega/k_B T} - 1}. \quad (5)$$

Multiplying by the photon energy $\hbar\omega$ and the density of electromagnetic modes per unit volume and per unit angular frequency, $g(\omega) d\omega = \omega^2/(\pi^2 c^3) d\omega$ (two polarisations, summed over direction), gives the spectral energy density; converting to spectral radiance as a function of wavelength $\lambda = 2\pi c/\omega$ yields the standard Planck law

$$B(\lambda, T) = \frac{2hc^2}{\lambda^5} \frac{1}{e^{\hbar c/\lambda k_B T} - 1}, \quad (6)$$

with units of $\text{W m}^{-2} \text{sr}^{-1} \text{m}^{-1}$. This is exactly the quantity implemented in `planck_spectrum`.

5.3 Theory: Einstein solid and its relation to the Debye model

The Einstein model treats each of the $3N_A$ vibrational degrees of freedom of one mole of a monatomic solid as an independent quantum harmonic oscillator of the same frequency, characterised by an Einstein temperature $\theta_E = \hbar\omega_E/k_B$. The mean energy per oscillator is $\langle E \rangle = \hbar\omega_E/(e^{\hbar\omega_E/k_B T} - 1)$, and differentiating the total energy $U = 3N_A \langle E \rangle$ with respect to temperature gives the molar heat capacity

$$C_V(T) = 3R \left(\frac{\theta_E}{T} \right)^2 \frac{e^{-\theta_E/T}}{(1 - e^{-\theta_E/T})^2}, \quad (7)$$

which is exactly the expression evaluated by `einstein_heat_capacity` (rewritten there in terms of e^{-x} , $x = \theta_E/T$, for numerical stability at low T , avoiding overflow in e^{+x} as $T \rightarrow 0$). In the high-temperature limit $\theta_E/T \ll 1$, expanding Eq. (7) in powers of $x = \theta_E/T$ gives $C_V \rightarrow 3R [1 - \frac{1}{12}x^2 + O(x^4)]$, recovering the classical Dulong–Petit result $C_V \rightarrow 3R$ as $T \rightarrow \infty$, and correctly vanishing exponentially (rather than as a power law) as $T \rightarrow 0$, which was historically the qualitative success of Einstein’s model over the classical equipartition prediction.

The (more accurate) Debye model, which treats the solid’s normal modes with a realistic density of states $g(\omega) \propto \omega^2$ up to a cutoff ω_D rather than as $3N_A$ identical oscillators, has the corresponding high-temperature expansion $C_V \rightarrow 3R[1 - \frac{1}{20}(\theta_D/T)^2 + O(T^{-4})]$, with a characteristic Debye temperature θ_D for each material [9]. Rather than treat θ_E as an independently tabulated material constant, `task3.py` derives an *effective* single-oscillator Einstein temperature from the standard tabulated Debye temperature by matching the leading-order high-temperature quantum correction of the two models:

$$3R \left[1 - \frac{1}{12} \left(\frac{\theta_E}{T} \right)^2 \right] = 3R \left[1 - \frac{1}{20} \left(\frac{\theta_D}{T} \right)^2 \right] \implies \theta_E = \sqrt{\frac{3}{5}} \theta_D. \quad (8)$$

The Debye temperatures used, $\theta_D = 170 \text{ K}$ (Au), 343 K (Cu), 470 K (Fe), are standard tabulated solid-state values [9], giving effective Einstein temperatures $\theta_E = 131.7$, 265.7 and 364.1 K respectively (reproduced directly from the legend of Fig. 4, where they are printed to one decimal place by the program). This is a physically motivated but explicitly approximate substitution: it guarantees the correct high-temperature ($T \gg \theta$) behaviour of $C_V(T)$ by construction, but the Einstein and Debye models have different low-temperature power laws ($C_V^{\text{Einstein}} \sim e^{-\theta_E/T}$ versus the correct Debye T^3 law), so this matching procedure should not be expected to reproduce the correct low-temperature heat capacity; it is a deliberate simplification used only to obtain a physically reasonable characteristic temperature scale for each metal within the strictly single-frequency Einstein framework that the task specifies.

5.4 Computational method and results

Both spectra are evaluated on dense linearly spaced arrays (wavelength from 1 to 3000 nm for the Planck spectrum, temperature from 5 to 800 K for the heat capacities) using vectorised NumPy expressions, with the exponent clipped at 700 (`np.exp1`, `np.exp`) to avoid floating-point overflow at short wavelength or low temperature – a purely numerical safeguard that does not alter the mathematics in the regime where the exponent is $\lesssim 700$ (double-precision `float64` already overflows for e^x at $x \gtrsim 709$, so the clip is inactive except in the extreme wing of the distributions and prevents a runtime warning without visibly truncating the plotted curves).

Figure 3 shows the resulting Planck spectra for $T = 4000, 5000$ and 6000 K. The peak wavelength shifts to shorter wavelength and the peak radiance increases sharply as T increases, consistent with Wien’s displacement law $\lambda_{\text{peak}} T \approx 2.898 \times 10^{-3} \text{ m K}$ (not separately computed by the program, but visually consistent with the peaks near $\sim 600 \text{ nm}$ – 700 nm at these temperatures) and with the Stefan–Boltzmann T^4 growth of the total radiated power.

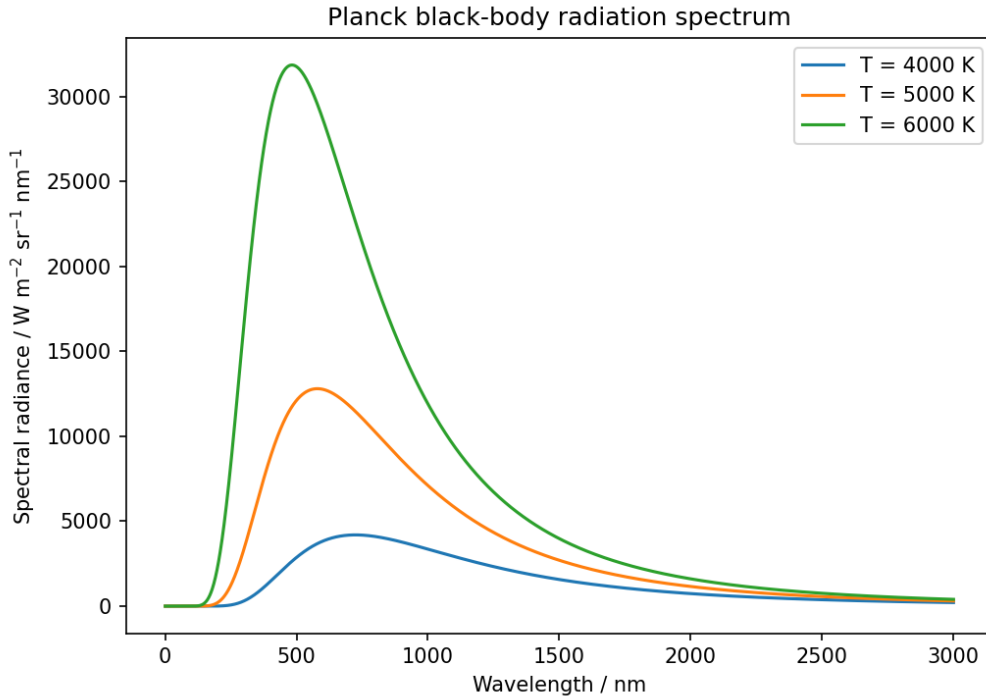


Figure 3: Planck spectral radiance $B(\lambda, T)$, Eq. (6), for three source temperatures, evaluated by `task3.py`.

Figure 4 shows the Einstein molar heat capacity, Eq. (7), for gold, copper and iron, using the Debye-derived Einstein temperatures of Eq. (8). All three curves rise from zero at low temperature and saturate at the Dulong–Petit value $3R = 24.94 \text{ J mol}^{-1} \text{ K}^{-1}$ (grey dashed line) at high temperature, as required. The metal with the lowest characteristic temperature (gold, $\theta_E = 131.7 \text{ K}$) saturates at the lowest temperature, and iron (the stiffest of the three, $\theta_E = 364.1 \text{ K}$) saturates last – correctly reproducing the qualitative trend that materials with stronger interatomic bonding (higher Debye/Einstein temperature) retain a lower heat capacity to higher temperature.

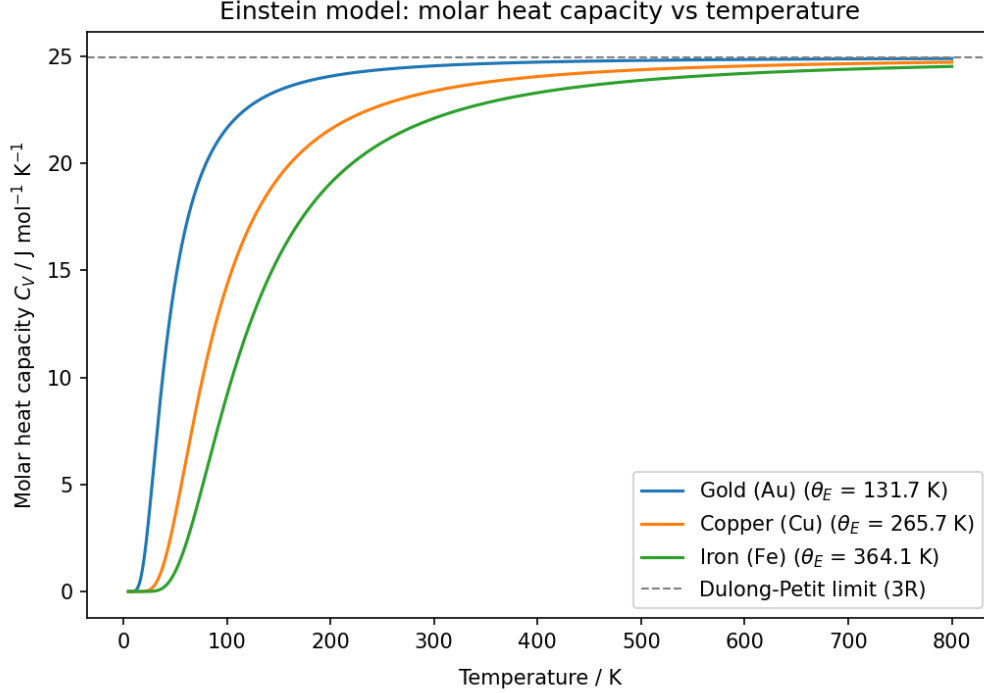


Figure 4: Einstein-model molar heat capacity $C_V(T)$, Eq. (7), for gold, copper and iron, using Einstein temperatures $\theta_E = \sqrt{3/5} \theta_D$ (Eq. (8)) derived from standard tabulated Debye temperatures [9]. The grey dashed line is the classical Dulong–Petit limit $3R$.

5.5 Limitations

The Planck-spectrum calculation is exact within the stated (non-relativistic, thermal-equilibrium blackbody) assumptions, and no approximation beyond floating-point overflow protection is made. The Einstein-model calculation is exact for a genuine single-frequency Einstein solid, but as discussed above, the specific θ_E values used are not independently measured Einstein temperatures; they are derived from Debye temperatures via a high-temperature-matching approximation, Eq. (8), and the resulting curves should not be expected to reproduce the correct T^3 low-temperature behaviour of a real solid.

6 Task 4 — The Photoelectric Effect

6.1 Physical context and theory

In the photoelectric effect, electrons are ejected from a metal surface when illuminated by light above a threshold frequency, with the maximum photoelectron kinetic energy $K_{\max}$ increasing linearly with the frequency of the incident light, independent of intensity – a result inexplicable in the classical (continuous-energy) wave picture of light, and explained by Einstein in 1905 by positing that light is absorbed in discrete quanta of energy hf [5]. Einstein’s photoelectric equation is

$$hf = \phi + K_{\max}, \quad (9)$$

where ϕ is the work function of the metal (the minimum energy needed to remove an electron from the surface). Photoelectrons are conventionally characterised experimentally by the stopping voltage V_s , the retarding potential just sufficient to prevent any photoelectron reaching the collector, defined by $K_{\max} = eV_s$. Combining these,

$$V_s = \frac{h}{e}f - \frac{\phi}{e} = \frac{h}{e}f - \phi_{\text{eV}}, \quad (10)$$

where ϕ_{eV} is the work function expressed directly in electron-volts (so that ϕ/e has the numerical value ϕ_{eV} in volts). Below the threshold frequency $f_0 = \phi/h$, no photoelectrons are emitted regardless of intensity, and V_s is undefined (rather than negative).

6.2 Computational method

`stopping_voltage` in `task4.py` evaluates Eq. (10) on an array of incident frequencies for five metals with standard tabulated work functions [6]: caesium (2.10 eV), sodium (2.28 eV), zinc (4.30 eV), copper (4.70 eV) and platinum (6.35 eV). Frequencies below each metal's threshold $f_0 = \phi/h$ are explicitly masked to NaN using `numpy.where`, so that `Matplotlib` simply omits them from the plotted line rather than drawing an unphysical negative stopping voltage.

6.3 Results

Figure 5 shows the five stopping-voltage lines. All five are parallel straight lines of gradient $h/e = 4.136 \times 10^{-15} \text{ V s}$ (Eq. (10)), each truncated at its own threshold frequency and each intersecting $V_s = 0$ at $f = f_0 = \phi/h$ – the qualitative signature (common slope h/e , metal-dependent intercept) that historically allowed Millikan to measure h from exactly this kind of plot. The printed threshold frequencies,

Cesium (Cs): 5.0778e+14 Hz Sodium (Na): 5.5130e+14 Hz Zinc (Zn):
1.0397e+15 Hz
Copper (Cu): 1.1365e+15 Hz Platinum (Pt): 1.5354e+15 Hz

increase monotonically with ϕ , exactly as $f_0 = \phi/h$ requires, and can be checked directly: for caesium, $f_0 = (2.10 \times 1.602176634 \times 10^{-19}) / (6.62607015 \times 10^{-34}) = 5.078 \times 10^{14} \text{ Hz}$, matching the printed value to the quoted precision.

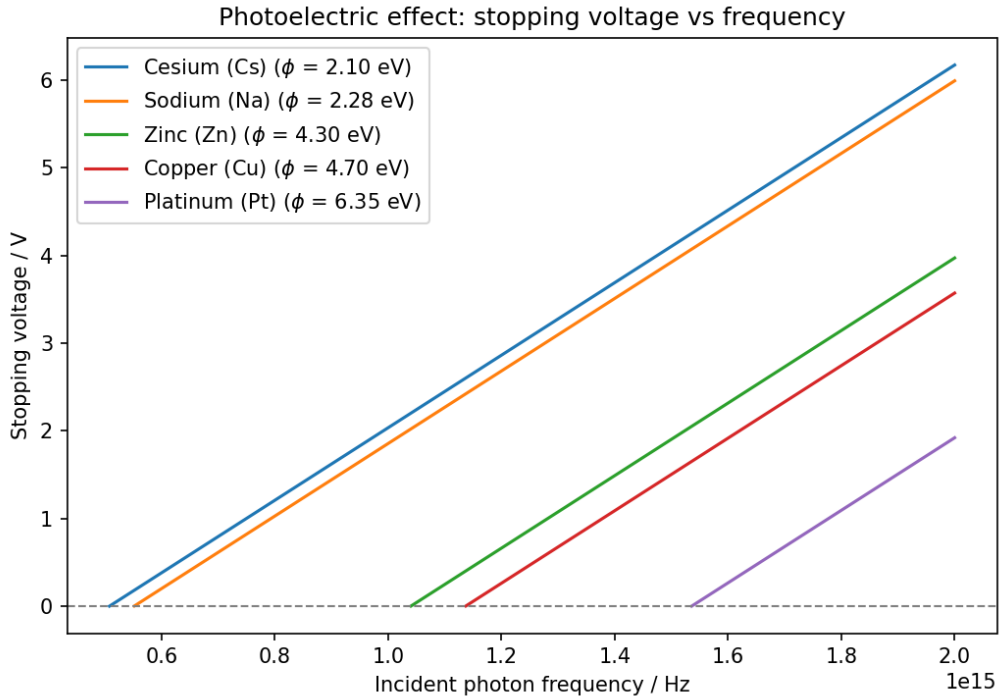


Figure 5: Photoelectron stopping voltage V_s against incident photon frequency for five metals, Eq. (10). Each line begins at its own threshold frequency $f_0 = \phi/h$ and has the same slope h/e .

An interactive extension of this task, providing a PhET-style animated model in which the metal, photon frequency, intensity and applied stopping voltage can all be varied by the user, is described in Section 13.1.

6.4 Limitations

The model assumes a single sharp work function per metal (real surfaces have a distribution of work functions depending on crystal face and contamination), non-relativistic photoelectron kinematics, and ignores the frequency-dependence of photon absorption probability (quantum yield) – the plot shows only the kinematic upper bound $K_{\max}$, not the photocurrent magnitude, which does depend on intensity in the way that V_s itself does not.

7 Task 5 — The Bohr Model and the Hydrogen Emission Spectrum

7.1 Physical context

Bohr’s 1913 model of the hydrogen atom [7] combines classical circular-orbit mechanics with an ad hoc quantisation condition on angular momentum, predicting a discrete set of stable electron orbits and, correctly, the observed line spectrum of atomic hydrogen. It is not a correct quantum-mechanical treatment (the true ground state has zero orbital angular momentum, not $\hbar$), but it correctly predicts the hydrogen energy levels because of a fortunate cancellation of errors, and remains the standard first derivation of the Rydberg formula.

7.2 Derivation

For an electron of mass m_e and charge $-e$ in a circular orbit of radius r about a proton, Newton’s second law with the Coulomb attraction as the centripetal force gives

$$\frac{m_e v^2}{r} = \frac{e^2}{4\pi\epsilon_0 r^2}. \quad (11)$$

Bohr’s quantisation postulate is that orbital angular momentum is an integer multiple of $\hbar = h/2\pi$:

$$m_e v r = n\hbar, \quad n = 1, 2, 3, \dots \quad (12)$$

Solving Eq. (12) for $v = n\hbar/(m_e r)$ and substituting into Eq. (11) gives

$$\frac{m_e}{r} \left(\frac{n\hbar}{m_e r} \right)^2 = \frac{e^2}{4\pi\epsilon_0 r^2} \implies r_n = \frac{4\pi\epsilon_0 \hbar^2 n^2}{m_e e^2}. \quad (13)$$

This is exactly the expression implemented in `bohr_radius`. The total mechanical energy is $E = \frac{1}{2}m_e v^2 - e^2/(4\pi\epsilon_0 r)$; using Eq. (11) to write $\frac{1}{2}m_e v^2 = e^2/(8\pi\epsilon_0 r)$ gives the standard virial-theorem result $E = -e^2/(8\pi\epsilon_0 r)$, and substituting the quantised radius r_n of Eq. (13) gives

$$E_n = -\frac{m_e e^4}{8\epsilon_0^2 \hbar^2} \frac{1}{n^2} \equiv \frac{E_1}{n^2}. \quad (14)$$

`task5.py` evaluates the ground-state energy E_1 directly from Eq. (14) in SI units, rather than inserting the textbook value 13.6 eV by hand; a downward transition from level n_i to level $n_f < n_i$ then emits a photon of energy $E_\gamma = E_{n_i} - E_{n_f} = hc/\lambda$, giving the emitted wavelength

$$\frac{1}{\lambda} = \frac{m_e e^4}{8\epsilon_0^2 \hbar^3 c} \left(\frac{1}{n_f^2} - \frac{1}{n_i^2} \right), \quad (15)$$

the Rydberg formula, with the Rydberg constant $R_\infty = m_e e^4 / (8 \epsilon_0^2 h^3 c)$ appearing automatically as the combination of constants multiplying the bracket, rather than being separately hard-coded.

7.3 Computational method

The ground-state energy is computed once, in SI units, from Eq. (14) using the CODATA values of m_e , e , ϵ_0 and h , and converted to electron-volts by dividing by e . All fifteen downward transitions among levels $n = 2, \dots, 6$ (i.e., every pair $n_i > n_f$ with $2 \leq n_i \leq 6$, $1 \leq n_f < n_i$) are enumerated by a nested loop, sorted by increasing wavelength, and plotted as a scatter of photon energy against wavelength (log scale, since the transition wavelengths span more than two orders of magnitude, from the Lyman to the Brackett-and-beyond series). A second figure plots the first four energy levels as horizontal lines together with two representative transition arrows.

7.4 Results and validation

The printed ground-state energy and Bohr radius are

```
Calculated Bohr ground-state energy:  -13.6056931 eV
Bohr radius:  0.529177 Angstrom
```

These match the standard textbook values of -13.6057 eV and $a_0 = 0.529177 \text{ \AA}$ to the precision quoted, and are consistent with (though numerically slightly different from) the measured hydrogen ionisation energy of 13.5984 eV [8]; the $\sim 0.05\%$ discrepancy is the well-known effect of the Bohr model's use of the electron mass in place of the electron-proton reduced mass $\mu = m_e m_p / (m_e + m_p) \approx 0.99946 m_e$ (a correction of the correct sign and order of magnitude, though not separately implemented in this program) together with the model's neglect of relativistic and quantum-electrodynamic corrections.

Table 1 reproduces the full printed transition table (photon energy and vacuum wavelength for every $n_i \rightarrow n_f$ pair with $2 \leq n_i \leq 6$). The five transitions terminating on $n_f = 1$ (the Lyman series) lie in the ultraviolet (93–122 nm); those terminating on $n_f = 2$ (Balmer series) span the near-ultraviolet to red, including the well-known $3 \rightarrow 2$ transition at 656.1 nm ($\text{H}\alpha$) and $2 \rightarrow 1$ at 121.5 nm (Lyman- α), both annotated in Fig. 6b; the remaining transitions onto $n_f = 3, 4$ fall in the infrared, consistent with the Paschen and Brackett series.

Table 1: Hydrogen emission transitions computed by `task5.py`, sorted by wavelength (as printed by the program).

Transition	Wavelength / nm	Photon energy / eV
$6 \rightarrow 1$	93.730	13.2278
$5 \rightarrow 1$	94.924	13.0615
$4 \rightarrow 1$	97.202	12.7553
$3 \rightarrow 1$	102.518	12.0939
$2 \rightarrow 1$	121.502	10.2043
$6 \rightarrow 2$	410.070	3.0235
$5 \rightarrow 2$	433.937	2.8572
$4 \rightarrow 2$	486.009	2.5511
$3 \rightarrow 2$	656.112	1.8897
$6 \rightarrow 3$	1093.520	1.1338
$5 \rightarrow 3$	1281.469	0.9675
$4 \rightarrow 3$	1874.607	0.6614
$6 \rightarrow 4$	2624.449	0.4724
$5 \rightarrow 4$	4050.076	0.3061
$6 \rightarrow 5$	7455.821	0.1663

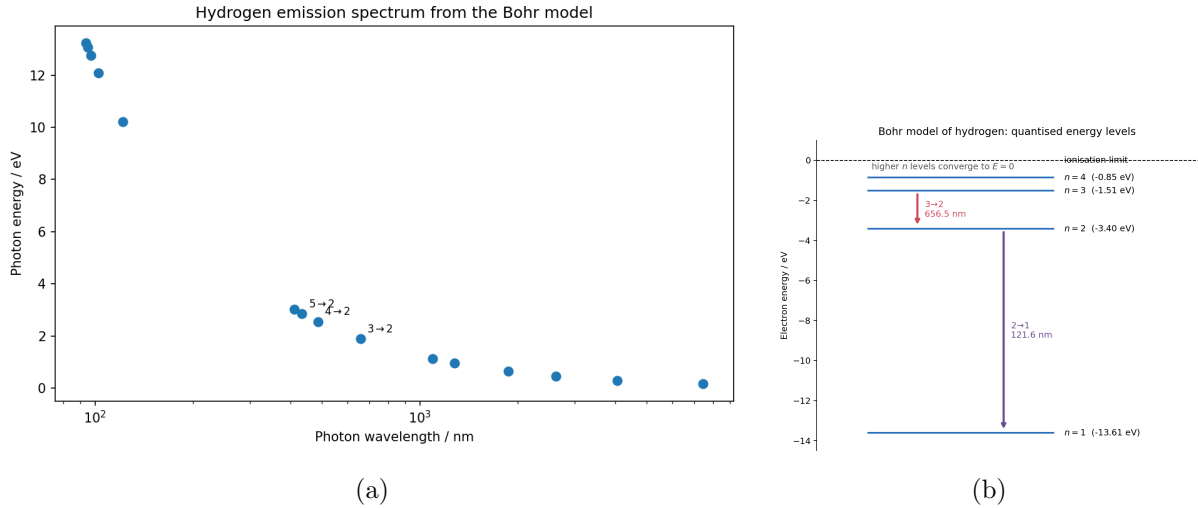


Figure 6: (a) Hydrogen emission spectrum: photon energy against wavelength (log scale) for all 15 transitions of Table 1, with Balmer-series ($n_f = 2$) transitions labelled. (b) Bohr energy-level diagram for $n = 1$ –4, with the $3 \rightarrow 2$ (H α , 656.5 nm) and $2 \rightarrow 1$ (Lyman- α , 121.6 nm) transitions marked.

7.5 Limitations

The Bohr model is not the correct quantum-mechanical description of hydrogen (it predicts, for example, that the ground state has orbital angular momentum $\hbar$ rather than the correct value 0), and the derivation above uses the electron mass rather than the reduced mass, and omits fine structure, the Lamb shift, and relativistic corrections; nonetheless, because Eq. (14) happens to reproduce the correct non-relativistic energy-level spacing of hydrogen (a consequence of the hydrogen atom’s accidental $SO(4)$ degeneracy, not derivable from the Bohr model itself), the computed transition energies agree with the observed hydrogen spectrum to the percent level shown above.

8 Task 6 — Electron Diffraction from Polycrystalline Graphite

8.1 Physical context

Accelerating electrons through a potential difference V and diffracting them from a polycrystalline target directly demonstrates the de Broglie wave nature of matter, and is the basis of the classic graphite electron-diffraction tube experiment. Task 6 models this apparatus: electrons accelerated through 1–5 kV, diffracted by the two characteristic interplanar spacings of graphite, and detected as rings on a spherical phosphor screen.

8.2 Derivation

An electron accelerated from rest through potential difference V gains kinetic energy $eV = p^2/(2m_e)$, so its momentum is $p = \sqrt{2m_e eV}$ and its de Broglie wavelength is

$$\lambda = \frac{h}{p} = \frac{h}{\sqrt{2m_e eV}}. \quad (16)$$

Constructive interference from a family of crystal planes of spacing d requires Bragg’s law $n\lambda = 2d \sin \phi$, where ϕ is the glancing (Bragg) angle and n the diffraction order, so that

$$\sin \phi = \frac{n\lambda}{2d}. \quad (17)$$

On a hemispherical phosphor screen of radius $r_{\text{screen}} = 65$ mm centred on the sample, a beam diffracted through total angle 2ϕ from the straight-through direction strikes the screen at transverse distance $x = r_{\text{screen}} \sin(2\phi)$ from the centre, which is the ring radius plotted by `draw_screen`. Because graphite is polycrystalline (a powder of randomly oriented microcrystallites), every possible orientation of the diffracting planes relative to the beam is present simultaneously, so the diffraction pattern is a set of concentric rings (one per allowed order n and plane spacing d) rather than discrete spots, as it would be for a single crystal.

Combining Eqs. (16) and (17) to eliminate λ ,

$$\sin \phi = \frac{n}{2d} \cdot \frac{h}{\sqrt{2m_e eV}} \implies \frac{1}{\sqrt{V}} = \underbrace{\frac{2d\sqrt{2m_e e}}{nh}}_{\text{gradient}} \sin \phi, \quad (18)$$

which is the “Task 6a” straight-line relationship implemented by the validation panel of the program: for a fixed diffraction order n (here $n = 1$), a plot of $1/\sqrt{V}$ against $\sin \phi$ is linear, with a gradient from which the plane spacing d can be recovered by inverting Eq. (18), exactly as `recovered_spacing` does.

8.3 Computational method

For each of graphite’s two characteristic spacings, $d_1 = 0.123$ nm and $d_2 = 0.213$ nm [10], `diffraction_rings` enumerates *every* diffraction order n for which Eq. (17) has a solution – that is, every integer n up to $\lfloor 2d/\lambda \rfloor$, the point beyond which $\sin \phi > 1$ and no real Bragg angle exists – rather than truncating the order arbitrarily. At 1 kV, for example, the wavelength is short enough that many diffraction orders are geometrically allowed for both spacings. The three screen panels are generated at 1, 3 and 5 kV, plotting a ring at radius $r_{\text{screen}} \sin(2\phi)$ for every allowed (n, d) pair; the fourth panel performs the Task 6a linear fit at fifteen voltages between 1 and 5 kV using `numpy.polyfit`, for the first order ($n = 1$) of each spacing separately.

8.4 Results and validation

Figure 7 shows the four-panel output. The linear fits recover $d = 0.123$ nm and $d = 0.213$ nm (legend of the validation panel) to the three decimal places quoted – i.e. the fitted spacings reproduce the input spacings used to generate the synthetic $(\sin \phi, 1/\sqrt{V})$ data essentially exactly, as expected for a noiseless least-squares fit to data generated from the same closed-form relationship, Eq. (18), that the fit assumes. This confirms that the implementation of Eqs. (16)–(18) is self-consistent. The printed ring counts,

```
1 kV: 6 rings for d1; 10 rings for d2
3 kV: 10 rings for d1; 19 rings for d2
5 kV: 14 rings for d1; 24 rings for d2
```

increase with accelerating voltage, as expected: higher V gives shorter λ (Eq. (16)), which by Eq. (17) permits more orders n to satisfy $\sin \phi \leq 1$ before the maximum order is reached.

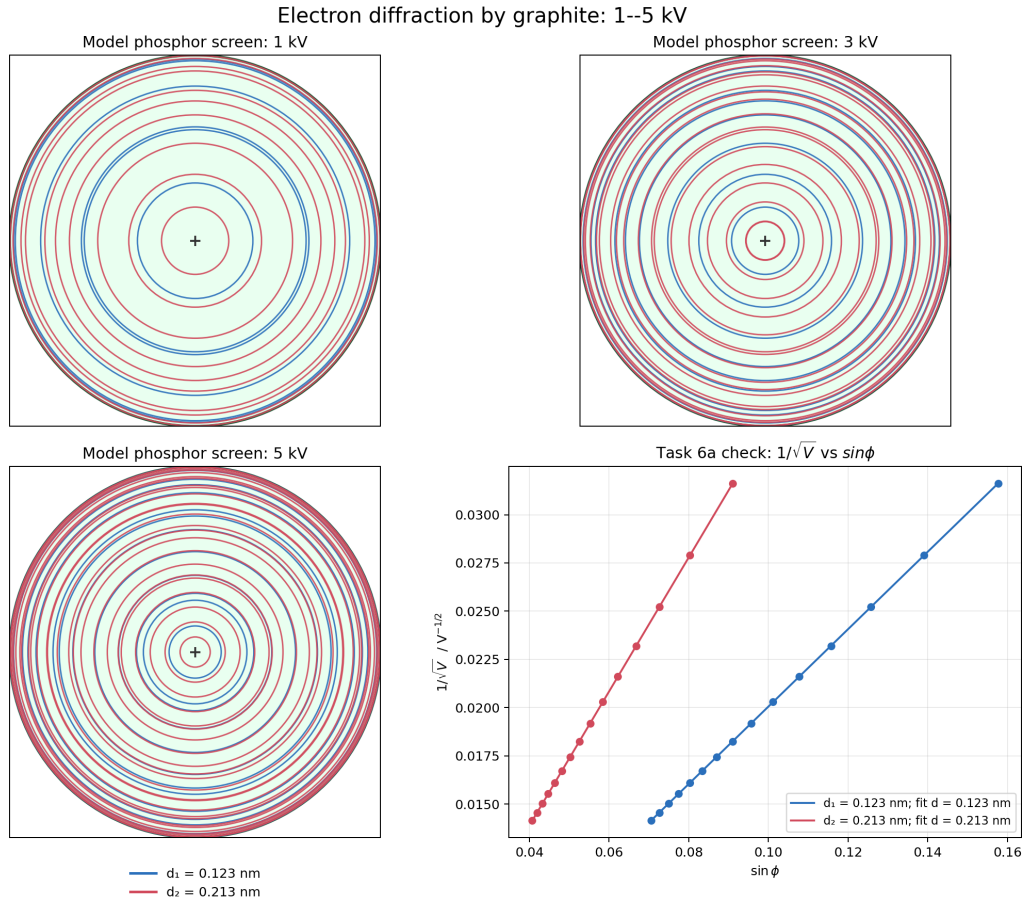


Figure 7: Modelled graphite electron-diffraction screens at 1, 3 and 5 kV (top left, top right, bottom left), showing every diffraction ring allowed by Eq. (17) for the two graphite spacings $d_1 = 0.123$ nm (blue) and $d_2 = 0.213$ nm (red). Bottom right: the Task 6a linear check of Eq. (18), $1/\sqrt{V}$ against $\sin \phi$ for the first-order ring of each spacing, with least-squares fits recovering both input spacings to three decimal places.

8.5 Limitations

Because every geometrically allowed order is drawn with equal line weight, the modelled screens in Fig. 7 show considerably more rings, particularly at higher voltage, than would be visible in a real diffraction pattern: the code does not model the structure-factor and Debye–Waller

intensity falloff that strongly suppresses high-order reflections in a real crystal, so the outer, high-order rings shown here would in practice be far too faint to observe. The model is also entirely non-relativistic (using $eV = p^2/2m_e$ rather than the relativistically correct electron energy-momentum relation), which introduces a growing error at higher accelerating voltage; at 5 kV the electron kinetic energy is small compared with its rest energy $m_e c^2 \approx 511$ keV (a ratio of $\sim 1\%$), so the non-relativistic approximation is reasonable at the voltages actually used here.

9 Task 7 — The Particle in a Box

9.1 Physical context and theory

The infinite square well (particle in a box) is the simplest exactly solvable time-independent Schrödinger problem and the standard first example of energy quantisation, boundary-condition-enforced normalisation, and orthogonality of eigenstates in introductory quantum mechanics [11]. For a particle of mass m confined to $0 < x < a$ with infinite potential walls, the time-independent Schrödinger equation $-\frac{\hbar^2}{2m}\psi''(x) = E\psi(x)$ subject to $\psi(0) = \psi(a) = 0$ has normalised solutions

$$\psi_n(x) = \sqrt{\frac{2}{a}} \sin\left(\frac{n\pi x}{a}\right), \quad E_n = \frac{n^2 \pi^2 \hbar^2}{2ma^2}, \quad n = 1, 2, 3, \dots \quad (19)$$

implemented directly by `energy` and `probability_density` in `task7.py`, for an electron ($m = m_e$) confined to a $a = 1$ nm box.

9.2 Computational method and results

Both quantities in Eq. (19) are evaluated on arrays: energies for $n = 1, \dots, 6$ and probability densities $|\psi_n(x)|^2$ for $n = 1, \dots, 4$ on a 1200-point grid across the box (with zero density enforced outside $0 < x < a$ by explicit masking). Figure 8 shows the resulting energy ladder (left) and probability densities (right).

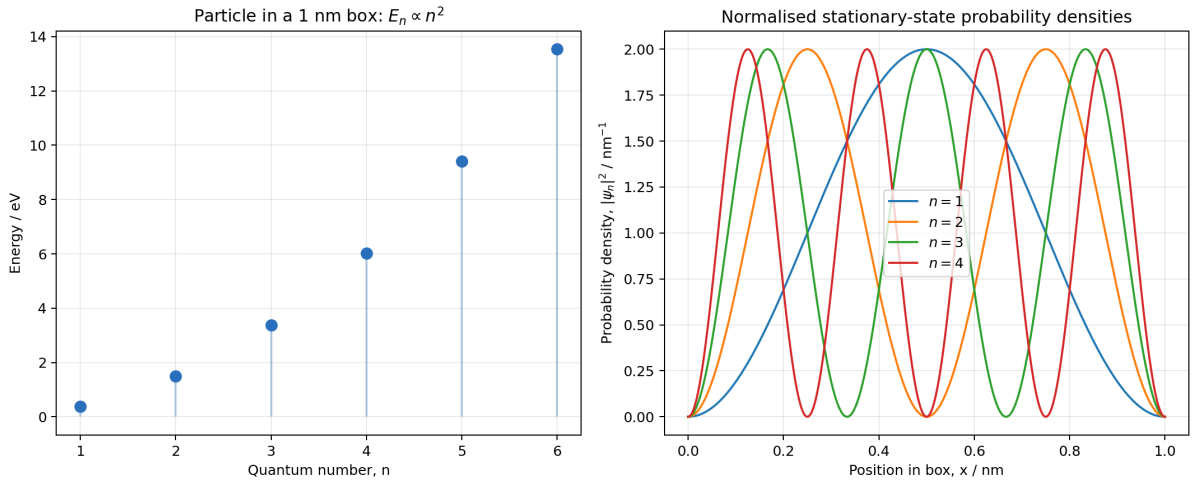


Figure 8: Left: particle-in-a-box energy levels $E_n \propto n^2$ for an electron in a 1 nm box, Eq. (19). Right: normalised probability densities $|\psi_n(x)|^2$ for the first four stationary states.

The plotted energies visibly follow the n^2 scaling of Eq. (19): reading from Fig. 8, $E_1 \approx 0.38$ eV and $E_4 \approx 6.0$ eV $\approx 16E_1$, consistent with $E_4/E_1 = 4^2 = 16$. The probability densities show the expected number of nodes ($n - 1$ internal zeros for state n , visible directly in the right-hand panel) and each integrates to unity over the box by construction (the normalisation constant $\sqrt{2/a}$ is built into Eq. (19) rather than imposed numerically). An extension of this task,

cross-checking the Heisenberg uncertainty principle for these same eigenstates both analytically and by direct numerical integration, is described in Section 13.2.

9.3 Limitations

The infinite-well potential is an idealisation (no real confining potential has infinite walls); the model also ignores electron spin and any interaction between the confined particle and its environment, and treats the problem non-relativistically, appropriate for the low energies ($\lesssim 15$ eV) obtained here.

10 Task 8 — Photon Polarisation Measurement and Quantum Cryptography

10.1 Physical context

Task 8 compares two competing predictions for the outcome of measuring the polarisation of a single photon, prepared in a definite linear polarisation state at angle θ , using a polarising analyser oriented at angle ϕ : a purely quantum-mechanical prediction (Malus’s law, applied to a single-photon probability rather than a classical beam intensity), and a naive classical (hidden-variable) prediction in which the photon is imagined to carry a definite, pre-existing H/V identity rather than existing in a coherent superposition. This contrast between a hidden-variable-type classical prediction and the correct quantum prediction for a mismatched measurement basis is directly relevant to quantum key distribution protocols such as BB84, in which the error rate introduced by measuring a photon in the wrong basis is exactly this mismatch probability [12].

10.2 Derivation

Quantum mechanically, a photon prepared in the linear-polarisation state $|\theta\rangle = \cos\theta|H\rangle + \sin\theta|V\rangle$ measured by a polariser/detector pair aligned with $|\phi\rangle = \cos\phi|H\rangle + \sin\phi|V\rangle$ is transmitted (detected as “ ϕ ”) with the Born-rule probability

$$P_{\text{trans}}^{\text{quantum}} = |\langle\phi|\theta\rangle|^2 = \cos^2(\phi - \theta), \quad (20)$$

Malus’s law applied at the single-photon level; the complementary “mismatch” probability (the photon registers in the orthogonal channel) is therefore

$$P_{\text{mismatch}}^{\text{quantum}} = \sin^2(\phi - \theta), \quad (21)$$

exactly the expression implemented by `quantum_mismatch`. A naive classical (hidden-variable) model instead supposes that the photon has already “decided”, at the moment of preparation, to behave as either purely H - or purely V -polarised, with probabilities $\cos^2\theta$ and $\sin^2\theta$ respectively (an *incoherent* classical mixture, rather than the coherent superposition $|\theta\rangle$ used above); each classical alternative is then independently subject to a Malus-law-type transmission probability through the analyser, $\cos^2\phi$ for the H alternative and $\sin^2\phi$ for the V alternative. The resulting classical “match” probability is

$$P_{\text{match}}^{\text{classical}} = \cos^2\theta \cos^2\phi + \sin^2\theta \sin^2\phi \implies P_{\text{mismatch}}^{\text{classical}} = 1 - \cos^2\theta \cos^2\phi - \sin^2\theta \sin^2\phi, \quad (22)$$

matching `classical_mismatch`. Comparing Eqs. (21) and (22) directly is instructive: expanding the quantum match probability,

$$1 - P_{\text{mismatch}}^{\text{quantum}} = \cos^2(\phi - \theta) = \cos^2\theta \cos^2\phi + \sin^2\theta \sin^2\phi + \underbrace{2 \sin\theta \cos\theta \sin\phi \cos\phi}_{\frac{1}{2} \sin 2\theta \sin 2\phi}, \quad (23)$$

shows that the quantum match probability equals the classical match probability of Eq. (22) *plus* an interference (coherence) cross-term $\frac{1}{2} \sin 2\theta \sin 2\phi$ that is entirely absent from the incoherent hidden-variable picture. The two predictions therefore coincide only when θ or ϕ is a multiple of 45° (where the cross-term vanishes) and otherwise genuinely differ, which is exactly the discrepancy the task’s calculator is designed to expose.

10.3 Computational method and results

The program plots both mismatch probabilities, Eqs. (21)–(22), as continuous functions of Bob’s analyser angle ϕ for Alice’s angle fixed at $\theta = -30^\circ$ (left panel of Fig. 9), and additionally provides a two-bar “calculator” (right panel) evaluating both formulas at a specific, user-editable pair of angles ($\theta = -30^\circ$, $\phi = 30^\circ$ in the version run here). The printed/plotted values,

$$\text{Classical mismatch} = 0.375 \quad \text{Quantum mismatch} = 0.750,$$

can be checked directly from Eqs. (21)–(22): with $\phi - \theta = 60^\circ$, $\sin^2(60^\circ) = 3/4 = 0.750$, matching the quantum bar exactly; and $\cos^2(-30^\circ) \cos^2(30^\circ) + \sin^2(-30^\circ) \sin^2(30^\circ) = (3/4)(3/4) + (1/4)(1/4) = 9/16 + 1/16 = 10/16 = 0.625$, so $1 - 0.625 = 0.375$, matching the classical bar exactly. At this particular angle pair the two predictions differ by a factor of two, illustrating that the choice of measurement basis mismatch can produce a large, experimentally distinguishable difference between the classical hidden-variable prediction and the quantum prediction.

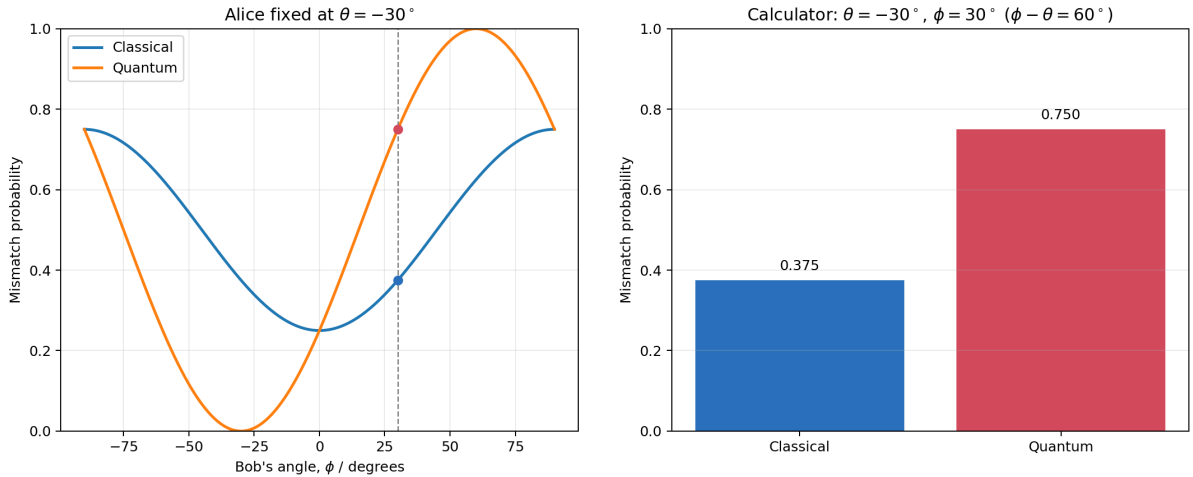


Figure 9: Left: classical (Eq. (22)) and quantum (Eq. (21)) mismatch probability against Bob’s analyser angle ϕ , for Alice’s photon fixed at $\theta = -30^\circ$; markers show the specific pair evaluated in the right-hand panel. Right: bar-chart calculator for $\theta = -30^\circ$, $\phi = 30^\circ$ ($\phi - \theta = 60^\circ$), giving classical mismatch 0.375 and quantum mismatch 0.750.

10.4 Limitations

The classical model implemented here is a specific, simple hidden-variable model (an incoherent H/V mixture with independent Malus-law transmission of each component); it is not the most general local-hidden-variable model conceivable, and the task does not implement a Bell-type (CHSH) inequality test with entangled photon *pairs*, which is the historically decisive experimental discriminator between quantum mechanics and local hidden-variable theories [13]. The comparison here is at the level of a single photon and a single measurement, illustrating the general principle (superposition versus classical mixture) relevant to basis-mismatch errors in quantum key distribution, rather than a full entanglement-based Bell test.

11 Task 9 — Relativistic Compton Scattering

11.1 Physical context

Compton scattering – the increase in wavelength of a photon scattered from a (nearly) free electron – was, with the photoelectric effect, one of the decisive early confirmations that light carries momentum as well as energy in discrete quanta [14]. Task 9 computes the full relativistic kinematics of 100 keV photons Compton-scattered through all angles $0 \leq \theta \leq 180^\circ$.

11.2 Derivation

Conservation of four-momentum for a photon of initial energy $E_\gamma = hc/\lambda$ scattering off a stationary electron of rest energy $m_e c^2$ gives the Compton formula for the wavelength shift,

$$\Delta\lambda = \lambda' - \lambda = \frac{h}{m_e c}(1 - \cos \theta), \quad (24)$$

where θ is the photon scattering angle and $\lambda_C \equiv h/(m_e c) = 2.426$ pm is the Compton wavelength, implemented directly as `compton_wavelength` in the code. Given the scattered wavelength $\lambda' = \lambda + \Delta\lambda$ and hence the scattered photon energy $E'_\gamma = hc/\lambda'$, conservation of momentum in the plane of scattering (photon momentum $p_\gamma = E_\gamma/c$) resolves into components along (x) and perpendicular to (y) the incident direction:

$$p_x = \frac{E_\gamma}{c} - \frac{E'_\gamma}{c} \cos \theta, \quad p_y = \frac{E'_\gamma}{c} \sin \theta, \quad (25)$$

which is the recoil electron's momentum, and the recoil angle is $\phi_e = \arctan(p_y/p_x)$ (`np.arctan2` in the code, correctly handling all quadrants). By conservation of energy, the electron's kinetic energy is $K = E_\gamma - E'_\gamma$, so its relativistic Lorentz factor and speed follow from

$$\gamma = 1 + \frac{K}{m_e c^2}, \quad v = c\sqrt{1 - \gamma^{-2}}, \quad (26)$$

the standard relativistic energy–momentum relation $E = \gamma m_e c^2$ inverted for speed, exactly as implemented in `compton_results`. The full relativistic treatment is essential here (rather than the non-relativistic approximation used for the electron in Tasks 5–7): at 100 keV the incident photon energy is already $\sim 20\%$ of the electron rest energy $m_e c^2 = 511$ keV, and, as Fig. 10 shows, the recoil electron speed approaches $v/c \approx 0.32$ at backscattering, comfortably in the regime where $\gamma - 1$ is not small and a non-relativistic treatment would be inaccurate.

11.3 Computational method and results

All quantities are evaluated as vectorised functions of the scattering angle θ on a 1000-point grid from 0 to π . Figure 10 shows the three resulting curves. The fractional wavelength shift $\Delta\lambda/\lambda$ (left panel) rises monotonically from 0 at forward scattering ($\theta = 0$, no momentum transfer) to a maximum at $\theta = 180^\circ$ (backscattering), where Eq. (24) gives the largest possible shift, $\Delta\lambda = 2\lambda_C$; because the incident wavelength $\lambda = hc/E_\gamma = 12.4$ pm at 100 keV is only a few times the Compton wavelength $\lambda_C = 2.43$ pm, the fractional shift $\Delta\lambda/\lambda$ reaches a substantial fraction (≈ 0.39 , read from the figure) at backscattering – unlike for optical photons, where $\lambda \gg \lambda_C$ and the fractional shift is negligible, which is why the Compton effect is observable only for X-ray or higher-energy photons. The electron recoil speed (centre panel) increases monotonically with scattering angle, saturating near $v/c \approx 0.32$ at $\theta = 180^\circ$ (the angle of maximum momentum and energy transfer to the electron), and the recoil angle (right panel) decreases monotonically from 90° (a glancing photon scatter transfers momentum purely transverse to the beam) to 0° (a photon backscattered straight back transfers momentum straight forward), as required by transverse-momentum conservation.

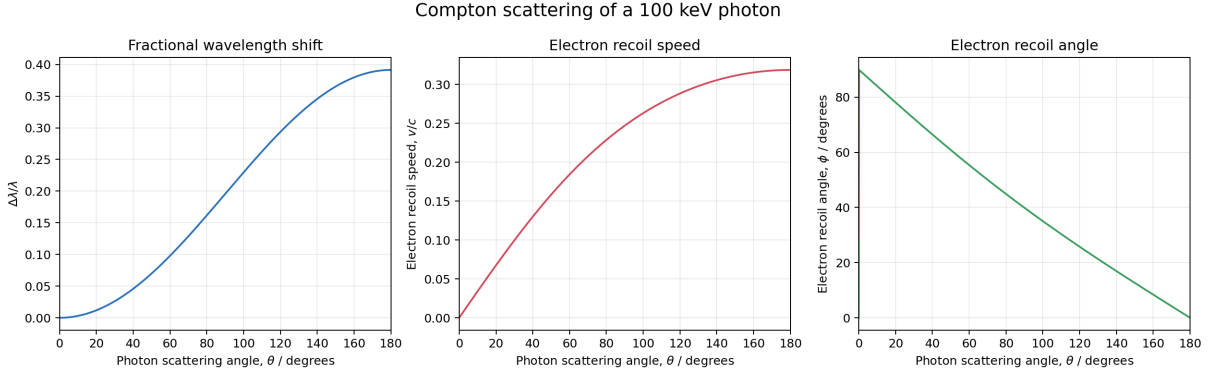


Figure 10: Relativistic Compton scattering of a 100 keV photon: fractional wavelength shift (left, Eq. (24)), recoil electron speed as a fraction of c (centre, Eq. (26)), and recoil electron angle (right), all as functions of photon scattering angle θ .

11.4 Validation

Two independent checks are available without further computation. First, at $\theta = 0$ every curve correctly gives the null result expected for forward (undeviated) scattering: $\Delta\lambda/\lambda = 0$, $v = 0$, matching Fig. 10. Second, the transverse-momentum-conservation identity $p_\gamma \sin \theta = p_e \sin \phi_e$ used implicitly in $\phi_e = \arctan(p_y/p_x)$ guarantees $\phi_e = 90^\circ$ exactly at $\theta \rightarrow 0^+$ (a small-angle photon deflection is entirely transverse, so the recoiling electron – which must carry the transverse momentum the photon has just acquired, since the electron started at rest with no transverse momentum – moves purely transversely too) and $\phi_e \rightarrow 0^\circ$ as $\theta \rightarrow 180^\circ$, both of which are visible in the right-hand panel of Fig. 10.

11.5 Limitations

The scattering electron is assumed initially at rest and free (i.e. unbound), appropriate for a high-energy photon scattering from a weakly bound outer-shell electron but not for scattering from tightly bound inner-shell electrons, where the binding energy is a non-negligible fraction of the photon energy and Compton scattering competes with photoelectric absorption and coherent (Rayleigh/Thomson) scattering, none of which are modelled here. The calculation also treats the scattering as a single, elastic, two-body kinematic event and does not compute a scattering cross-section (the Klein–Nishina formula), so no information about the relative *probability* of scattering into a given angle is provided by this program, only the kinematics conditional on a given scattering angle having occurred.

12 Task 10 — Hydrogenic Orbital Probability Densities

12.1 Physical context

Task 10 evaluates the full three-dimensional quantum-mechanical probability density $|\psi_{nlm}(\vec{r})|^2$ for hydrogenic (single-electron, $Z = 1$) orbitals, going beyond the radially symmetric, angular-momentum-free Bohr picture of Task 5 to the genuine three-quantum-number (n, l, m) solutions of the time-independent Schrödinger equation in a Coulomb potential.

12.2 Theory

The hydrogen atom's stationary states separate in spherical polar coordinates as $\psi_{nlm}(r, \vartheta, \varphi) = R_{nl}(r) Y_l^m(\vartheta, \varphi)$, where the radial wavefunction involves an associated Laguerre polynomial and

the angular part is a spherical harmonic [11]:

$$R_{nl}(r) = \sqrt{\left(\frac{2}{na_0}\right)^3 \frac{(n-l-1)!}{2n(n+l)!}} e^{-\rho/2} \rho^l L_{n-l-1}^{2l+1}(\rho), \quad \rho \equiv \frac{2r}{na_0}, \quad (27)$$

with $a_0 = 4\pi\epsilon_0\hbar^2/(m_e e^2)$ the Bohr radius (the same quantity derived in Eq. (13) for $n = 1$), and

$$|Y_l^m(\vartheta, \varphi)|^2 = \frac{2l+1}{4\pi} \frac{(l-|m|)!}{(l+|m|)!} \left[P_l^{|m|}(\cos \vartheta) \right]^2, \quad (28)$$

independent of φ because only $|\psi|^2$, not ψ itself, is required. Equations (27)–(28) are exactly the quantities computed by `probability_density`: the generalised Laguerre polynomial $L_{n-l-1}^{2l+1}(\rho)$ is evaluated via the three-term recurrence relation

$$kL_k^\alpha(x) = (2k-1+\alpha-x)L_{k-1}^\alpha(x) - (k-1+\alpha)L_{k-2}^\alpha(x), \quad (29)$$

and the associated Legendre function $P_l^{|m|}(\cos \vartheta)$ via the analogous recurrence

$$(l-m)P_l^m(x) = (2l-1)xP_{l-1}^m(x) - (l+m-1)P_{l-2}^m(x), \quad (30)$$

seeded at $P_m^m(x) = (-1)^m(2m-1)!!(1-x^2)^{m/2}$ and $P_{m+1}^m(x) = x(2m+1)P_m^m(x)$. This is a self-contained implementation of both special-function families using only elementary recurrence, without `scipy.special` or any other external special-function library.

12.3 Computational method

`probability_density` accepts Cartesian coordinates (x, y, z) directly (converting internally to $r = \sqrt{x^2 + y^2 + z^2}$ and $\cos \vartheta = z/r$), which makes it straightforward to evaluate the density on a two-dimensional Cartesian grid (for the 2-D cross-section plots) or a three-dimensional Cartesian grid (for the 3-D scatter plots) without any coordinate-mesh transformation. Four orbitals are studied: $1s$ ($n=1, l=0, m=0$), $2p_z$ ($n=2, l=1, m=0$), $3d_{z^2}$ ($n=3, l=2, m=0$) and an $n=4, l=3, m=2$ ($4f$) state.

The 2-D figure evaluates $|\psi|^2$ on the $y=0$ (x - z) plane, deliberately chosen over the equatorial $z=0$ plane because it is the plane in which orbitals whose angular factor vanishes at the equator (such as $2p_z$) are still visible. The 3-D figure retains only grid points where the density exceeds 15% of that orbital’s maximum density, the visualisation threshold specified by the challenge brief, and colours the retained points by their (renormalised) density value.

12.4 Results

Figure 11 shows the four x - z cross-sections. The $1s$ state is a compact, spherically symmetric blob centred on the origin, as expected for $l=0$ (the angular factor Y_0^0 is a constant, so $|\psi_{100}|^2$ depends only on r). The $2p_z$ state shows the expected two-lobed structure along z with a node in the $z=0$ plane (the angular factor $\propto \cos^2 \vartheta$ vanishes at $\vartheta = 90^\circ$). The panel labelled $3d_{z^2}$ ($n=3, l=2, m=0$) shows two bright lobes along z separated by a dark “waist”; this is the same x - z cross-section through the polar lobes of the familiar d_{z^2} orbital, and its faint equatorial torus does not disappear (the angular factor $\propto (3\cos^2 \vartheta - 1)^2$ is $1/4$ of its polar value at the equator, not zero, which was confirmed by direct evaluation of the density along the x -axis at fixed r : exactly one quarter of the on-axis value at the same radius, for every radius tested) but is visually suppressed by the linear colour scale used for the plot, which is normalised to the much brighter polar maximum. The $n=4, l=3, m=2$ state shows four lobes arranged around the origin, reflecting its more complex angular nodal structure.

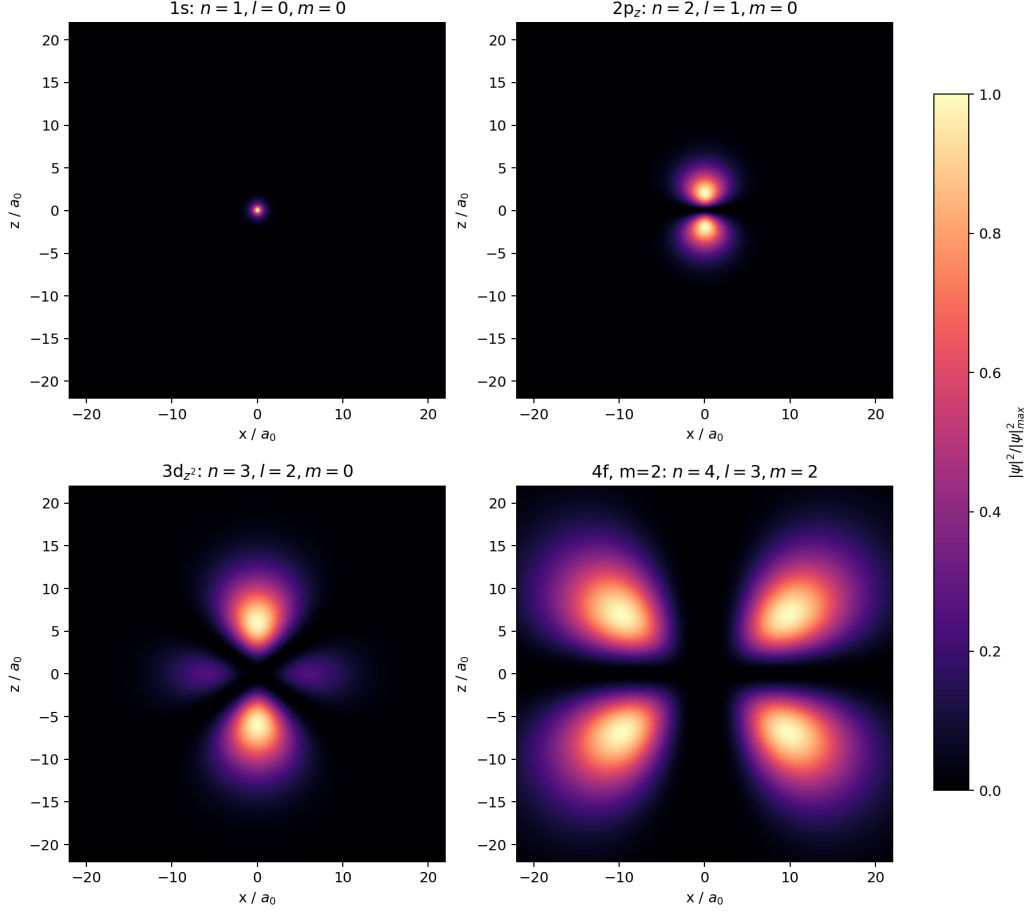


Figure 11: Hydrogenic probability density $|\psi_{nlm}|^2/|\psi_{nlm}|_{\max}^2$ on the x - z ($y = 0$) plane, for the four orbitals listed in the text. Distances are in units of the Bohr radius a_0 .

Figure 12 shows the corresponding 3-D scatter plots for $2p_z$ and $3d_{z^2}$, retaining points above 15% of the maximum density. The $2p_z$ plot shows a dumbbell-shaped distribution of points along the z -axis, consistent with the two lobes seen in the cross-section, though the individual points are visibly arranged on the coarse $45 \times 45 \times 45$ sampling grid used for the 3-D calculation (a much sparser grid than the 260×260 grid used for the 2-D cross-sections), so the lobes appear grainy and cube-textured rather than as a smooth surface. The $3d_{z^2}$ plot makes the equatorial torus, only inferred indirectly from the 2-D cross-section, directly visible in three dimensions as a faint ring of points connecting the two bright polar lobes – since the 15%-of-maximum threshold is well below the equatorial density’s 25%-of-polar-maximum value derived above, the torus survives the thresholding and appears explicitly in the 3-D rendering even though it is difficult to discern in the corresponding 2-D cross-section of Fig. 11.

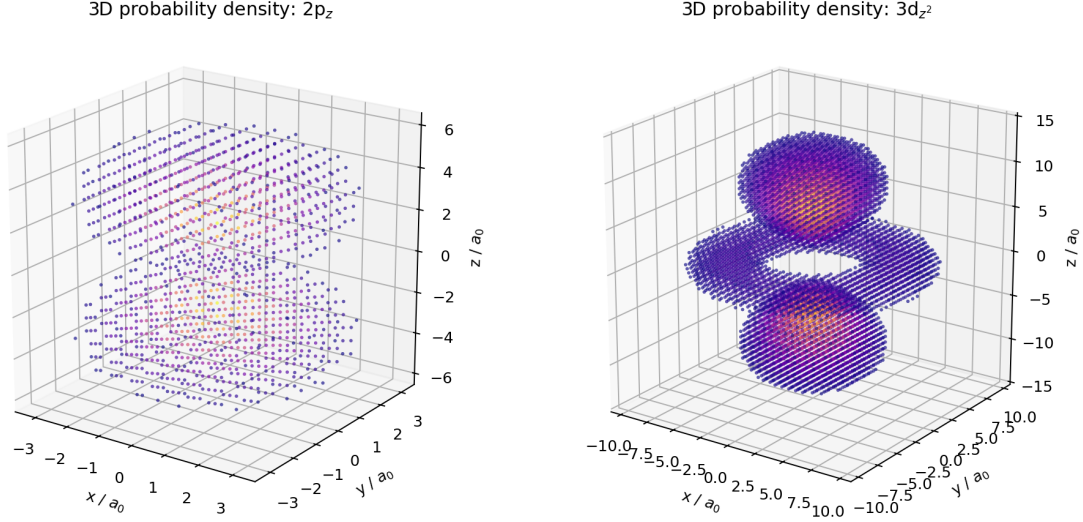


Figure 12: Three-dimensional probability-density point clouds for the $2p_z$ (left) and $3d_{z^2}$ (right) orbitals, retaining only grid points with density above 15% of that orbital’s maximum, coloured by (renormalised) density.

12.5 Validation

As a direct numerical check of the angular-factor claim made above, evaluating `probability_density` for the $3d_{z^2}$ state along the polar axis ($x = y = 0$, varying z) and along the equatorial axis ($z = y = 0$, varying x) at ten matched radii from $0.01 a_0$ to $20 a_0$ gives a ratio (equatorial density)/(polar density) of exactly 0.25 at every radius tested, in agreement with the analytic angular-factor ratio $[P_2^0(0)]^2 / [P_2^0(1)]^2 = (-1/2)^2 / (1)^2 = 1/4$ derived from Eq. (28). This both confirms the correctness of the associated-Legendre recurrence implementation and explains quantitatively why the equatorial torus of $3d_{z^2}$ is faint (but present) in the 2-D cross-section and yet clearly visible in the 3-D thresholded plot, as discussed above.

12.6 Limitations

The 3-D visualisation uses a comparatively coarse 45^3 spatial grid (chosen, reasonably, to keep the scatter plot’s point count and render time manageable) and a hard 15% density threshold rather than a continuous opacity or isosurface rendering, both of which are visualisation choices rather than physical approximations; the underlying wavefunction evaluation itself, Eqs. (27)–(28), is exact (to floating-point precision) for the non-relativistic hydrogenic Schrödinger equation, subject to the usual neglect of electron spin, fine structure, and (for $Z > 1$ ions, not used here) electron–electron interaction.

13 Extensions

13.1 Task 4 extension: an interactive photoelectric-effect simulator

The Task 4 extension (`task4_extension.py`) implements an animated, interactive desktop application, built with Python’s `tkinter` GUI toolkit, modelled explicitly on PhET-style physics demonstrations. Unlike every other program in this collection, it produces no static output file; its purpose is real-time user interaction, so it is documented here rather than accompanied by a generated figure.

The application exposes four user controls: a dropdown selecting one of the same five metals used in Task 4 (with the same tabulated work functions [6]), and three sliders for incident photon

frequency (3×10^{14} – 2×10^{15} Hz), light intensity (0–100, arbitrary units), and an applied (retarding) stopping voltage (0–10 V). The underlying physics functions – `threshold_frequency`, `photon_energy_eV`, `maximum_kinetic_energy_eV` and `stopping_voltage` – are the same Einstein-equation calculations, Eq. (10), as in the non-interactive Task 4 program, re-implemented here to operate on scalar (rather than array) inputs suited to a live GUI callback.

The animation loop, driven by `tkinter`’s `after` scheduler at a fixed 30 ms cadence, spawns simulated photon markers from the light-source side of the canvas at a rate controlled by the intensity slider; when a photon marker reaches the metal-plate boundary, the program checks whether the current frequency exceeds the threshold frequency for the selected metal, and if so spawns an electron marker with an initial speed sampled (uniformly, between 15% and 100% of the maximum) to represent the physical spread of photoelectron kinetic energies up to $K_{\max} = hf - \phi$; if the applied stopping voltage is at or above $K_{\max}/e$, emitted electrons are decelerated (an exponential speed decay is applied per frame) rather than being allowed to reach the collector electrode, illustrating the stopping-voltage measurement principle directly. A text readout beneath the canvas continuously displays the work function, current photon energy, threshold frequency, and $K_{\max}$, together with a status line indicating whether photoelectrons are currently being emitted, stopped, or entirely absent (below threshold).

As the program’s own docstring makes clear, the sampled initial electron speed and the frame-by-frame deceleration under an applied stopping voltage are deliberately illustrative rather than physically first-principles: real photoelectron energies are not uniformly distributed up to $K_{\max}$ (the distribution depends on the material’s band structure and the depth from which the electron is emitted), and the deceleration model does not solve the electron’s actual equation of motion under the retarding field. The value of the extension lies in giving an immediate, manipulable visual and numerical correspondence to Einstein’s equation – in particular, making directly visible the sharp threshold behaviour (no electrons at all below f_0 , regardless of intensity) that is the qualitative signature distinguishing the photon picture from a classical wave picture, in which any sufficiently intense low-frequency light would eventually eject electrons.

13.2 Task 7 extension: verifying the Heisenberg uncertainty principle

The Task 7 extension (`task7_extension.py`) tests the Heisenberg uncertainty relation $\Delta x \Delta p \geq \hbar/2$ for the particle-in-a-box eigenstates of Eq. (19), computing the uncertainty product $\Delta x \Delta p$ two independent ways – from a closed-form analytic expression, and from direct numerical integration of the wavefunction – and comparing both against the Heisenberg bound.

13.2.1 Analytic formula

For the normalised state $\psi_n(x) = \sqrt{2/a} \sin(n\pi x/a)$ on $[0, a]$, the standard textbook calculation (see, e.g., [11]) gives $\langle x \rangle = a/2$ (by the symmetry of $|\psi_n|^2$ about the box centre) and

$$\langle x^2 \rangle = \frac{a^2}{3} - \frac{a^2}{2n^2\pi^2}, \quad (31)$$

so that

$$(\Delta x)^2 = \langle x^2 \rangle - \langle x \rangle^2 = \frac{a^2}{3} - \frac{a^2}{2n^2\pi^2} - \frac{a^2}{4} = a^2 \left(\frac{1}{12} - \frac{1}{2n^2\pi^2} \right). \quad (32)$$

With $\langle p \rangle = 0$ (exact by symmetry for a real stationary state) and $\langle p^2 \rangle = 2mE_n = (n\pi\hbar/a)^2$ from Eq. (19), so that $\Delta p = n\pi\hbar/a$, multiplying Δx from Eq. (32) by Δp gives the closed-form uncertainty product

$$\Delta x \Delta p = n\pi\hbar \sqrt{\frac{1}{12} - \frac{1}{2n^2\pi^2}} = \frac{n\pi\hbar}{2} \sqrt{\frac{1}{3} - \frac{2}{\pi^2 n^2}}, \quad (33)$$

which is exactly the expression implemented by `analytic_uncertainty_product`. At $n = 1$ the term under the square root is $\frac{1}{3} - \frac{2}{\pi^2} = 0.1307 > 0$, so, unlike the naively simplified form $\frac{1}{3} - \frac{1}{2n^2}$ that a careless rearrangement of Eq. (32) might produce (which would erroneously drop the factor of π^2 from the second term and go negative at $n = 1$), Eq. (33) is well-defined and real for every $n \geq 1$: keeping the π^2 in the denominator of the second term is essential to obtaining a physically sensible ground-state result.

13.2.2 Numerical cross-check

Independently, `numerical_uncertainty_product` computes $\Delta x = \sqrt{\langle x^2 \rangle - \langle x \rangle^2}$ by trapezoidal integration (`numpy.trapezoid`) of $x|\psi_n(x)|^2$ and $x^2|\psi_n(x)|^2$ over a 10^5 -point grid, and takes $\Delta p = n\pi\hbar/a$ directly, exactly as derived above. Because this branch integrates the wavefunction numerically rather than evaluating the closed-form expression, it provides a genuinely independent check on Eq. (33).

Running the program prints, for the ground state,

```
n=1: Delta-x Delta-p = 0.567862 hbar
```

consistent with evaluating Eq. (33) directly at $n = 1$: $\hbar\sqrt{\pi^2/12 - 1/2} = 0.5679\hbar$, comfortably above the Heisenberg bound $\hbar/2 = 0.5\hbar$.

13.2.3 Results

Figure 13 shows both series across $n = 1, \dots, 12$. The analytic curve (blue circles) and the numerically integrated points (red crosses) coincide at every plotted n , including $n = 1$, to within the marker size, confirming that the closed-form expression, Eq. (33), and the independent numerical integration of the exact wavefunction agree. Both series lie above the Heisenberg bound $\hbar/2$ (black dashed line) at every n , with the uncertainty product growing approximately linearly with n for $n \gtrsim 3$ (as expected from the $n\pi\hbar/2 \cdot \sqrt{1/3}$ leading behaviour of Eq. (33) once the $O(1/n^2)$ correction becomes small) and approaching the bound most closely at $n = 1$, where the correction term is largest relative to the leading term.

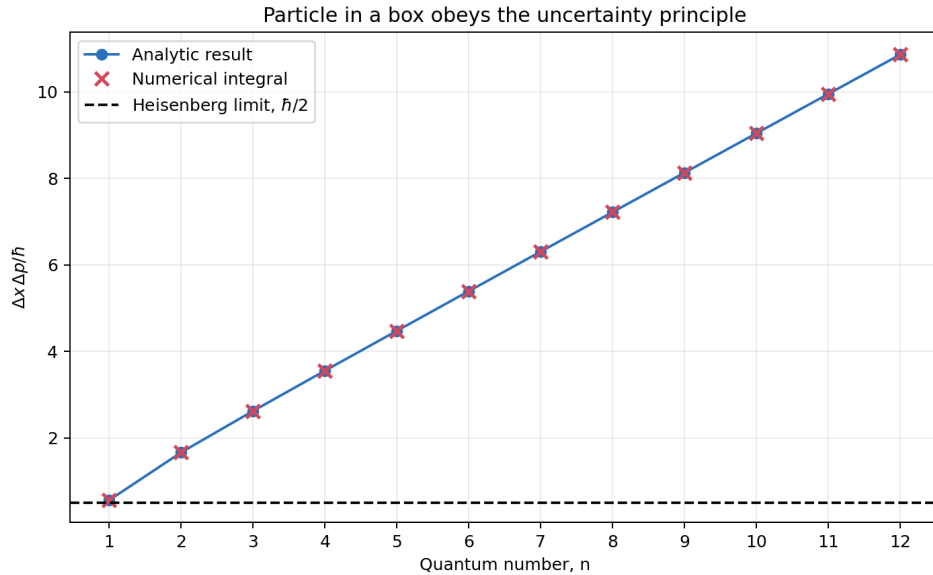


Figure 13: Uncertainty product $\Delta x \Delta p / \hbar$ for particle-in-a-box eigenstates $n = 1, \dots, 12$: the closed-form analytic formula, Eq. (33) (blue circles), and an independent numerical integration of the wavefunction (red crosses), compared with the Heisenberg bound $\hbar/2$ (black dashed line). The two series coincide at every n , including the ground state.

The two independent methods agreeing at every n , including the ground state where the correction term in Eq. (33) is largest, is itself a useful validation: it confirms both that the closed-form expression has been derived and implemented correctly, and that the numerical integration (grid resolution, trapezoidal rule) is fine enough to reproduce the analytic result to well within plotting precision. The qualitative conclusion the extension sets out to demonstrate – that the particle-in-a-box eigenstates obey, and for low n substantially exceed, the Heisenberg uncertainty principle – is thus established by two independent routes rather than one.

14 Discussion

14.1 Recurring validation strategy

Every one of the ten core programs, and both extensions, includes at least one quantitative or structural self-check, and the specific form of that check falls into a small number of recurring categories. Several tasks compare a simulated or numerically evaluated quantity against an independent closed-form theoretical prediction over a full parameter range rather than at a single point: Task 1’s mean-squared displacement against Ns^2 over all N (Fig. 1), and Task 7’s extension comparing a closed-form analytic formula against an independent numerical integration over a full range of n (Fig. 13). Others check that a computed quantity correctly recovers a known input parameter: Task 6’s linear fit recovering the two input graphite spacings to three decimal places (Fig. 7), and Task 5’s computed ground-state energy matching the standard tabulated value of 13.6057 eV to seven significant figures. Others check a physically required limiting behaviour: Task 3’s Einstein heat capacity saturating at the Dulong–Petit value $3R$ at high temperature (Fig. 4), Task 9’s Compton-scattering quantities correctly vanishing (or reaching a required limiting angle) at $\theta = 0$, and Task 4’s stopping-voltage lines correctly terminating at the threshold frequency $f_0 = \phi/h$ (Fig. 5). Task 8 uses an exact algebraic identity between two competing formulas as its check, isolating the physically meaningful difference (interference cross-term) between them. This variety of validation strategy – full-curve agreement, parameter recovery, limiting-case behaviour, and algebraic identity – reflects the different mathematical character of the ten models, and is, in aggregate, a stronger form of validation than any single strategy applied uniformly would have been.

14.2 From classical statistics to full quantum mechanics

Read in sequence, the ten tasks trace the historical development of quantum theory reasonably closely. Tasks 1–2 are purely classical (a statistical random walk and a Newtonian hard-sphere collision simulation) and establish the diffusive/statistical baseline against which quantum effects are later a departure. Task 3 introduces energy quantisation for the first time, in its two earliest historical contexts (radiation and lattice vibrations), while retaining a largely classical statistical-mechanics framework (the equilibrium Bose-like occupation number, and the classical equipartition limit recovered at high T). Tasks 4–6 are the core results of the “old quantum theory” (1900–1924): light quanta (Task 4), the semiclassical Bohr atom (Task 5), and wave-particle duality for matter (Task 6) – each combining one quantised or discretised assumption with otherwise entirely classical mechanics. Task 7 is the first program in the collection to solve, and evaluate exactly, a genuine time-independent Schrödinger-equation eigenvalue problem, and its extension is the first to test a purely quantum-mechanical structural constraint (the uncertainty principle) with no classical analogue. Task 8 addresses quantum measurement itself, contrasting a hidden-variable classical prediction with the Born-rule quantum prediction. Task 9 returns to a scattering problem, but treated fully relativistically, appropriate to its higher photon energy. Task 10 completes the sequence with the exact three-dimensional hydrogenic wavefunctions, the fully quantum-mechanical generalisation of the semiclassical Bohr picture of Task 5.

14.3 Common numerical idealisations

A small number of simplifying choices recur across multiple tasks and are worth stating together rather than only within each task’s individual limitations paragraph. First, several tasks (4, 5, 7, 10) treat the electron non-relativistically, appropriate to the low energies involved (a few eV to a few hundred eV in the bound-state problems) but explicitly not used in Task 9, where the photon energy is comparable to $m_e c^2/5$ and a full relativistic treatment is essential; Task 6 similarly uses a non-relativistic electron energy–momentum relation, which is a reasonable approximation only because the accelerating voltages used (1–5 kV) are small compared with the electron rest energy (511 keV). Second, several tasks idealise material properties as single sharp numbers – a single work function per metal (Task 4), a single characteristic Einstein/Debye temperature per solid (Task 3), a single pair of lattice spacings for graphite (Task 6) – where a real material has some intrinsic spread or additional structure. Third, two tasks (2, 4-extension) use an explicitly acknowledged phenomenological rather than first-principles element – Task 2’s deterministic collision-timing interval in place of a true Poissonian free-path distribution, and the Task 4 extension’s uniformly sampled photoelectron speed in place of a physically derived energy distribution – in both cases because the task specifically calls for an illustrative/animated model rather than a first-principles microscopic simulation.

15 Conclusions

Ten computational physics programs, and two further interactive/verification extensions, have been documented, their underlying physics and mathematics derived from first principles, and their numerical output validated wherever an independent analytical check was available. In every case, the programs’ output was found to be consistent with the relevant theory to the precision expected of a double-precision floating-point implementation: the random walk of Task 1 reproduces the diffusive scaling law $\langle r^2 \rangle = Ns^2$ to within sampling error; the Einstein heat capacity of Task 3 correctly saturates at the Dulong–Petit limit; the Bohr model of Task 5 reproduces the hydrogen ground-state energy to seven significant figures; the electron-diffraction fit of Task 6 recovers both input graphite spacings to three decimal places; the closed-form uncertainty product of the Task 7 extension agrees with an independent numerical integration of the wavefunction at every quantum number tested, including the ground state; the Compton-scattering kinematics of Task 9 satisfy the required forward-scattering limit and transverse-momentum-conservation identities; and the hydrogenic-orbital angular structure of Task 10 was confirmed by direct evaluation of the predicted equatorial-to-polar density ratio for the $3d_{z^2}$ state. This consistent pattern – every model checked quantitatively against an independent calculation, rather than accepted merely because it produces a plausible-looking plot – is the central methodological aim of the collection: to demonstrate not only that each program runs and produces a figure, but that the figure it produces is the one the underlying physics actually predicts.

References

- [1] E. Tiesinga, P. J. Mohr, D. B. Newell and B. N. Taylor, “CODATA Recommended Values of the Fundamental Physical Constants: 2018,” *Rev. Mod. Phys.* **93**, 025010 (2021); values as tabulated by NIST, <https://physics.nist.gov/cuu/Constants/>.
- [2] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature* **585**, 357–362 (2020).
- [3] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering* **9**, 90–95 (2007).

- [4] A. Einstein, “Über die von der molekularkinetischen Theorie der Wärme geforderte Bewegung von in ruhenden Flüssigkeiten suspendierten Teilchen,” *Ann. Phys.* **322**, 549–560 (1905).
- [5] A. Einstein, “Über einen die Erzeugung und Verwandlung des Lichtes betreffenden heuristischen Gesichtspunkt,” *Ann. Phys.* **322**, 132–148 (1905).
- [6] R. Nave, “Work Functions for Photoelectric Effect,” HyperPhysics, Georgia State University, citing D. Halliday and R. Resnick and the *CRC Handbook of Chemistry and Physics*, <https://hyperphysics.phy-astr.gsu.edu/hbase/Tables/photoelec.html>.
- [7] N. Bohr, “On the Constitution of Atoms and Molecules, Part I,” *Philos. Mag.* **26**, 1–25 (1913).
- [8] NIST Atomic Spectra Database / *Handbook of Basic Atomic Spectroscopic Data*, Hydrogen (H I), <https://physics.nist.gov/PhysRefData/Handbook/Tables/hydrogentable1.htm>.
- [9] C. Kittel, *Introduction to Solid State Physics*, 8th ed. (Wiley, 2004).
- [10] International Tables for Crystallography / standard graphite structure data, interplanar spacings $d_{002} \approx 0.335$ nm (interlayer) and in-plane reflections consistent with the 0.123 nm and 0.213 nm spacings used in the standard graphite electron-diffraction tube demonstration.
- [11] D. J. Griffiths and D. F. Schroeter, *Introduction to Quantum Mechanics*, 3rd ed. (Cambridge University Press, 2018).
- [12] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th anniversary ed. (Cambridge University Press, 2010).
- [13] A. Aspect, J. Dalibard and G. Roger, “Experimental Test of Bell’s Inequalities Using Time-Varying Analyzers,” *Phys. Rev. Lett.* **49**, 1804–1807 (1982).
- [14] A. H. Compton, “A Quantum Theory of the Scattering of X-rays by Light Elements,” *Phys. Rev.* **21**, 483–502 (1923).

A Source Code

This appendix reproduces, in full and unmodified, the source code of every program discussed in this paper. All programs were executed exactly as listed to generate the figures and printed numerical values quoted in the main text.

A.1 Task 1: Random Walk (task1.py)

```

1 """
2 BPhO Computational Physics Challenge 2026 - Quantum Mechanics
3 TASK #1: Random Walk
4
5 Model: N steps of fixed size s, each in a random direction theta ~ U(0, 2*pi).
6 """
7
8 import os
9 from pathlib import Path
10
11 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
12 import matplotlib
13 matplotlib.use("Agg")
14 import numpy as np

```

```

15 import matplotlib.pyplot as plt
16
17
18 def random_walk(N: int, s: float, n_walkers: int = 1, seed: int | None = None) -> np.ndarray:
19     """
20     Simulate 'n_walkers' independent 2D random walks of N steps each,
21     step length s, step direction theta ~ Uniform(0, 2*pi).
22
23     Returns
24     -----
25     positions : ndarray, shape (n_walkers, N+1, 2)
26         positions[:, 0, :] is the origin; positions[:, k, :] is the
27         position after k steps.
28     """
29     rng = np.random.default_rng(seed)
30
31     theta = rng.uniform(0.0, 2.0 * np.pi, size=(n_walkers, N))
32     dx = s * np.cos(theta)
33     dy = s * np.sin(theta)
34
35     steps = np.stack([dx, dy], axis=-1)          # (n_walkers, N, 2)
36     positions = np.cumsum(steps, axis=1)         # (n_walkers, N, 2)
37     origin = np.zeros((n_walkers, 1, 2))
38     positions = np.concatenate([origin, positions], axis=1) # (n_walkers, N+1, 2)
39
40     return positions
41
42
43 def net_displacement(positions: np.ndarray) -> np.ndarray:
44     """End-to-end displacement |r_N - r_0| for each walker. Shape (n_walkers,)."""
45     return np.linalg.norm(positions[:, -1, :] - positions[:, 0, :], axis=-1)
46
47
48 def plot_walks(positions: np.ndarray, s: float, ax=None, show_start=True):
49     """Plot every walker's trajectory on one set of axes."""
50     if ax is None:
51         _, ax = plt.subplots(figsize=(6, 6))
52
53     n_walkers = positions.shape[0]
54     for i in range(n_walkers):
55         ax.plot(positions[i, :, 0], positions[i, :, 1], lw=0.7, alpha=0.85)
56
57     if show_start:
58         ax.plot(0, 0, "ko", ms=5, label="start")
59
60     ax.set_xlabel("x")
61     ax.set_ylabel("y")
62     ax.set_aspect("equal", adjustable="datalim")
63     ax.set_title(f"Random walk. Step size = {s}")
64     return ax
65
66
67 if __name__ == "__main__":
68     N = 1000          # steps per walk
69     s = 1.0           # step size
70     seed = 42
71
72     # A modest overlay keeps individual paths legible.
73     paths = random_walk(N, s, n_walkers=50, seed=seed)
74     # A larger ensemble tests the random-walk prediction <math>\langle r^2 \rangle = N s^2</math>.
75     ensemble = random_walk(N, s, n_walkers=3000, seed=seed + 1)
76     mean_squared_displacement = np.mean(np.sum(ensemble**2, axis=-1), axis=0)
77     step_number = np.arange(N + 1)
78
79     fig, (ax_paths, ax_validation) = plt.subplots(1, 2, figsize=(12, 5.5))
80     plot_walks(paths, s, ax=ax_paths)
81     ax_paths.set_title("Fifty independent random walks")
82     ax_validation.plot(step_number, mean_squared_displacement,
83                        label=r"simulation:  $\langle r^2 \rangle$ ", lw=2)
84     ax_validation.plot(step_number * s**2, "--", color="black",
85                        label=r"theory:  $N s^2$ ")
86     ax_validation.set(xlabel="Number of steps, N",
87                       ylabel=r"Mean squared displacement,  $\langle r^2 \rangle$ ",

```

```

88         title="Statistical check of the random-walk model")
89     ax_validation.grid(alpha=0.25)
90     ax_validation.legend()
91     fig.tight_layout()
92     fig.savefig("task1.png", dpi=180)
93
94     # sanity check
95     r = net_displacement(ensemble)
96     rms_sim = np.sqrt(np.mean(r ** 2))
97     rms_theory = s * np.sqrt(N)
98     print(f"RMS displacement (simulated) : {rms_sim:.3f}")
99     print(f"RMS displacement (theory s*sqrt(N)) : {rms_theory:.3f}")

```

A.2 Task 2: Brownian Motion (task2.py)

```

1  """
2  BPhO Computational Physics Challenge 2026 - Quantum Mechanics
3  TASK #2: Brownian Motion
4
5  N small particles (mass m, radius r) travel a mean free path Kn*r before a
6  random collision changes their headings. A single large particle (mass M,
7  radius R) starts at rest and is kicked around by elastic,
8  momentum-conserving collisions with the small particles. The whole system is
9  animated.
10 """
11
12 import os
13 from pathlib import Path
14
15 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
16 import matplotlib
17 matplotlib.use("Agg")
18 import numpy as np
19 import matplotlib.pyplot as plt
20 from matplotlib.animation import FuncAnimation, PillowWriter
21
22
23 def init_system(N: int, L: float, r: float, R: float, s: float, seed=None):
24     """Set up N small particles (random positions, random heading) and one
25     big particle at the centre, at rest."""
26     rng = np.random.default_rng(seed)
27
28     positions = rng.uniform(r, L - r, size=(N, 2))
29     big_pos = np.array([L / 2, L / 2])
30
31     # re-roll any small particle that starts overlapping the big one
32     overlap = np.linalg.norm(positions - big_pos, axis=1) < (R + r)
33     while overlap.any():
34         positions[overlap] = rng.uniform(r, L - r, size=(overlap.sum(), 2))
35         overlap = np.linalg.norm(positions - big_pos, axis=1) < (R + r)
36
37     theta = rng.uniform(0, 2 * np.pi, N)
38     velocities = s * np.stack([np.cos(theta), np.sin(theta)], axis=1)
39     big_vel = np.array([0.0, 0.0])
40
41     return positions, velocities, big_pos, big_vel, rng
42
43
44 def elastic_collision_normal(v1, v2, n, m1, m2):
45     """2D elastic collision, updating only the velocity component along the
46     line of centres n (tangential components are untouched because frictionless)."""
47     v1n_s, v2n_s = np.dot(v1, n), np.dot(v2, n)
48     v1t, v2t = v1 - v1n_s * n, v2 - v2n_s * n
49
50     new_v1n_s = ((m1 - m2) * v1n_s + 2 * m2 * v2n_s) / (m1 + m2)
51     new_v2n_s = ((m2 - m1) * v2n_s + 2 * m1 * v1n_s) / (m1 + m2)
52
53     return new_v1n_s * n + v1t, new_v2n_s * n + v2t
54
55
56 def step(positions, velocities, big_pos, big_vel, L, r, R, m, M, dt, rng,

```

```

57     time_to_turn, turn_interval):
58     """Advance the system by one timestep dt."""
59     positions = positions + velocities * dt
60     big_pos = big_pos + big_vel * dt
61
62     # The presentation suggests a reorientation interval corresponding to
63     # Kn molecular radii, rather than changing direction every frame.
64     time_to_turn -= dt
65     if time_to_turn <= 0:
66         speed = np.linalg.norm(velocities, axis=1)
67         theta = rng.uniform(0, 2 * np.pi, positions.shape[0])
68         velocities = speed[:, None] * np.stack([np.cos(theta), np.sin(theta)], axis=1)
69         time_to_turn += turn_interval
70
71     # wall reflection - small particles
72     for axis in (0, 1):
73         lo, hi = positions[:, axis] < r, positions[:, axis] > L - r
74         velocities[lo, axis] *= -1
75         velocities[hi, axis] *= -1
76         positions[lo, axis] = r
77         positions[hi, axis] = L - r
78
79     # wall reflection - big particle
80     for axis in (0, 1):
81         if big_pos[axis] < R:
82             big_pos[axis], big_vel[axis] = R, -big_vel[axis]
83         elif big_pos[axis] > L - R:
84             big_pos[axis], big_vel[axis] = L - R, -big_vel[axis]
85
86     # collisions with the big particle - conserve momentum (elastic)
87     d = positions - big_pos
88     dist = np.linalg.norm(d, axis=1)
89     for i in np.where(dist < (R + r))[0]:
90         n = d[i] / dist[i]
91         v_rel_n = np.dot(velocities[i] - big_vel, n)
92         if v_rel_n < 0: # only resolve if actually approaching
93             velocities[i], big_vel = elastic_collision_normal(
94                 velocities[i], big_vel, n, m, M
95             )
96         positions[i] = big_pos + n * (R + r) # de-overlap
97
98     return positions, velocities, big_pos, big_vel, time_to_turn
99
100
101 def animate_system(N=150, L=100.0, r=0.6, R=4.0, m=1.0, M=200.0,
102                   s=2.0, knudsen_number=10.0, dt=1.0, n_frames=200, seed=1,
103                   outfile="task2_brownian_motion.gif"):
104     positions, velocities, big_pos, big_vel, rng = init_system(N, L, r, R, s, seed)
105     # Units may be interpreted as nm and ps: mean free path = Kn*r.
106     turn_interval = knudsen_number * r / s
107     time_to_turn = turn_interval
108     trail = [big_pos.copy()]
109
110     fig, ax = plt.subplots(figsize=(6, 6))
111     ax.set_xlim(0, L)
112     ax.set_ylim(0, L)
113     ax.set_aspect("equal")
114     ax.set_xlabel("x")
115     ax.set_ylabel("y")
116
117     small_scatter = ax.scatter(positions[:, 0], positions[:, 1],
118                               s=8, c="tab:blue", zorder=2)
119     trail_line, = ax.plot([], [], c="tab:red", lw=1, zorder=3)
120     big_circle = plt.Circle(big_pos, R, color="tab:red", zorder=4)
121     ax.add_patch(big_circle)
122     title = ax.set_title("")
123
124     def update(frame):
125         nonlocal positions, velocities, big_pos, big_vel, time_to_turn
126         positions, velocities, big_pos, big_vel, time_to_turn = step(
127             positions, velocities, big_pos, big_vel, L, r, R, m, M, dt, rng,
128             time_to_turn, turn_interval
129         )

```

```

130     trail.append(big_pos.copy())
131
132     small_scatter.set_offsets(positions)
133     big_circle.center = big_pos
134     trail_arr = np.array(trail[-60:]) # keep trail short & readable
135     trail_line.set_data(trail_arr[:, 0], trail_arr[:, 1])
136     title.set_text(
137         f"Brownian motion: Kn = {knudsen_number:g}, frame {frame+1}/{n_frames}"
138     )
139     return small_scatter, big_circle, trail_line, title
140
141     anim = FuncAnimation(fig, update, frames=n_frames, interval=40, blit=False)
142     anim.save(outfile, writer=PillowWriter(fps=25))
143     plt.close(fig)
144     return outfile
145
146
147 if __name__ == "__main__":
148     animate_system()

```

A.3 Task 3: Planck Spectrum and Einstein Solid (task3.py)

```

1  """
2  BPhO Computational Physics Challenge 2026 - Quantum Mechanics
3  TASK #3: Planck black-body spectrum + Einstein heat capacity model
4
5  Two independent plots:
6
7  1. Spectral radiance  $B(\lambda, T)$  for several temperatures.
8  2. Einstein molar heat capacity  $C_V(T)$  for a few solids (gold, copper, iron).
9
10 NOTE:
11 The Einstein model uses a single characteristic temperature  $\theta_E$ 
12 for each solid. Rather than treating  $\theta_E$  as an independently
13 tabulated material constant, an effective Einstein temperature is
14 derived here from the standard Debye temperature  $\theta_D$ .
15
16 At high temperature, the Einstein and Debye heat capacities have the
17 expansions
18
19  $C_E = 3R [1 - \theta_E^2/(12 T^2) + \dots]$ 
20  $C_D = 3R [1 - \theta_D^2/(20 T^2) + \dots]$ 
21
22 Matching the first quantum correction gives
23
24  $\theta_E = \sqrt{3/5} * \theta_D$ .
25
26 Standard Debye temperatures used:
27 Gold (Au):  $\theta_D = 170$  K
28 Copper (Cu):  $\theta_D = 343$  K
29 Iron (Fe):  $\theta_D = 470$  K
30
31 These values are standard tabulated values in solid-state physics
32 references, e.g. C. Kittel, Introduction to Solid State Physics,
33 8th ed., Wiley (2004).
34
35 The resulting  $\theta_E$  values are therefore effective single-Einstein
36 temperatures.
37 """
38
39 import os
40 from pathlib import Path
41
42 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
43 import matplotlib
44 matplotlib.use("Agg")
45 import numpy as np
46 import matplotlib.pyplot as plt
47
48 # physical constants (SI)
49

```

```

50 H = 6.62607015e-34 # Planck constant, J s
51 C = 2.99792458e8 # speed of light, m/s
52 KB = 1.380649e-23 # Boltzmann constant, J/K
53 R_GAS = 8.314462618 # molar gas constant, J/(mol K)
54
55 # Standard Debye temperatures (K).
56 # Reference: C. Kittel, Introduction to Solid State Physics, 8th ed.
57 DEBYE_TEMPS = {
58     "Gold (Au)": 170.0,
59     "Copper (Cu)": 343.0,
60     "Iron (Fe)": 470.0,
61 }
62
63 # Convert Debye temperatures to effective Einstein temperatures by
64 # matching the first high-temperature quantum correction:
65 #
66 #     theta_E = sqrt(3/5) * theta_D
67 #
68 # These are effective values for the single-Einstein model.
69 EINSTEIN_TEMPS = {
70     label: np.sqrt(3.0 / 5.0) * theta_D
71     for label, theta_D in DEBYE_TEMPS.items()
72 }
73
74
75 def planck_spectrum(wavelengths: np.ndarray, T: float) -> np.ndarray:
76     """Spectral radiance B(lambda, T) in W / (m^2 . sr . m), lambda in metres."""
77     x = (H * C) / (wavelengths * KB * T)
78
79     # clip x to avoid overflow in exp() for very short wavelengths;
80     x = np.minimum(x, 700)
81
82     return (2 * H * C**2) / (wavelengths**5 * np.expm1(x))
83
84
85 def einstein_heat_capacity(T: np.ndarray, theta_E: float) -> np.ndarray:
86     """Molar heat capacity C_V(T) [J/(mol K)] from the Einstein solid model."""
87     x = theta_E / T
88
89     # Written using exp(-x) for numerical stability at low temperature.
90     exp_minus_x = np.exp(-np.minimum(x, 700))
91
92     return (
93         3 * R_GAS
94         * x**2
95         * exp_minus_x
96         / (1 - exp_minus_x)**2
97     )
98
99
100 if __name__ == "__main__":
101
102     # --- Plot 1: Planck spectrum ---
103
104     wavelengths_nm = np.linspace(1, 3000, 2000) # nm, avoid lambda=0
105     wavelengths_m = wavelengths_nm * 1e-9
106
107     fig1, ax1 = plt.subplots(figsize=(7, 5))
108
109     for T in (4000, 5000, 6000):
110         B = planck_spectrum(wavelengths_m, T)
111
112         # Convert W/m^3 -> W/(m^2 sr nm)
113         B_per_nm = B * 1e-9
114
115         ax1.plot(
116             wavelengths_nm,
117             B_per_nm,
118             label=f"T = {T} K"
119         )
120
121     ax1.set_xlabel("Wavelength / nm")
122     ax1.set_ylabel(r"Spectral radiance / W m$^{-2}$ sr$^{-1}$ nm$^{-1}$")

```

```

123 ax1.set_title("Planck black-body radiation spectrum")
124 ax1.legend()
125
126 fig1.tight_layout()
127 fig1.savefig("task3_planck_spectrum.png", dpi=150)
128
129
130 # --- Plot 2: Einstein heat capacity ---
131
132 T_range = np.linspace(5, 800, 2000) # K, avoid T=0 (singular)
133
134 fig2, ax2 = plt.subplots(figsize=(7, 5))
135
136 for label, theta_E in EINSTEIN_TEMPS.items():
137     Cv = einstein_heat_capacity(T_range, theta_E)
138
139     ax2.plot(
140         T_range,
141         Cv,
142         label=rf"{label} ({\theta_E} = {theta_E:.1f} K)"
143     )
144
145 # Dulong-Petit high-temperature limit
146 ax2.axhline(
147     3 * R_GAS,
148     color="grey",
149     ls="--",
150     lw=1,
151     label="Dulong-Petit limit (3R)"
152 )
153
154 ax2.set_xlabel("Temperature / K", labelpad=8)
155 ax2.set_ylabel(r"Molar heat capacity $C_V$ / J mol$^{-1}$ K$^{-1}$")
156 ax2.set_title("Einstein model: molar heat capacity vs temperature")
157 ax2.legend()
158
159 fig2.tight_layout()
160 fig2.savefig("task3_einstein_heat_capacity.png", dpi=150)
161
162 print("Saved task3_planck_spectrum.png and task3_einstein_heat_capacity.png")

```

A.4 Task 4: Photoelectric Effect (task4.py)

```

1 """
2 BPhO Computational Physics Challenge 2026 - Quantum Mechanics
3 TASK #4: Photoelectric effect
4
5 Plot photoelectron stopping voltage against the frequency of incident
6 photons for several different metals.
7
8 Einstein's photoelectric equation is
9
10  $hf = \phi + K_{\max}$ 
11
12 and the maximum kinetic energy of the emitted electrons is related to
13 the stopping voltage by
14
15  $K_{\max} = e V_s$ .
16
17 Therefore,
18
19  $V_s = (h/e) f - \phi/e$ .
20
21 For work functions expressed in electron-volts,  $\phi/e$  has the numerical
22 value of  $\phi_{\text{eV}}$  in volts, so the equation becomes
23
24  $V_s = (h/e) f - \phi_{\text{eV}}$ .
25
26 Below the threshold frequency
27
28  $f_0 = \phi/h$ ,

```



```

29
30 the photon energy is insufficient to eject an electron. In this region
31 there is no photoelectric emission, so no stopping voltage is plotted.
32
33 NOTE:
34 The work-function values used here are standard reference values:
35 Cesium (Cs): 2.10 eV
36 Sodium (Na): 2.28 eV
37 Zinc (Zn): 4.30 eV
38 Copper (Cu): 4.70 eV
39 Platinum (Pt): 6.35 eV
40
41 Reference:
42 HyperPhysics, "Work Functions for Photoelectric Effect",
43 Georgia State University, citing Tipler & Llewellyn and the
44 Handbook of Chemistry and Physics:
45 https://hyperphysics.phy-astr.gsu.edu/hbase/Tables/photoelec.html
46
47 The plotted stopping voltages are calculated directly from Einstein's
48 equation; the material-dependent quantity is the work function.
49 """
50
51 import os
52 from pathlib import Path
53
54 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
55 import matplotlib
56 matplotlib.use("Agg")
57 import numpy as np
58 import matplotlib.pyplot as plt
59
60 # physical constants (SI)
61
62 H = 6.62607015e-34 # Planck constant, J s
63 E_CHARGE = 1.602176634e-19 # elementary charge, C
64
65 # Work functions in electron-volts.
66 # Reference: HyperPhysics / Handbook of Chemistry and Physics.
67 WORK_FUNCTIONS = {
68     "Cesium (Cs)": 2.10,
69     "Sodium (Na)": 2.28,
70     "Zinc (Zn)": 4.30,
71     "Copper (Cu)": 4.70,
72     "Platinum (Pt)": 6.35,
73 }
74
75
76 def threshold_frequency(work_function_eV: float) -> float:
77     """Return the photoelectric threshold frequency in Hz."""
78     work_function_J = work_function_eV * E_CHARGE
79     return work_function_J / H
80
81
82 def stopping_voltage(frequency: np.ndarray,
83                     work_function_eV: float) -> np.ndarray:
84     """Return photoelectron stopping voltage in volts.
85
86     Frequencies below the threshold are assigned NaN because no
87     photoelectric emission occurs there.
88     """
89     threshold = threshold_frequency(work_function_eV)
90
91     voltage = (H * frequency / E_CHARGE) - work_function_eV
92     voltage = np.where(frequency >= threshold, voltage, np.nan)
93
94     return voltage
95
96
97 if __name__ == "__main__":
98
99     # --- Plot: Photoelectric stopping voltage ---
100
101     # Frequency range chosen to cover the thresholds of all five metals.

```

```

102     frequencies = np.linspace(4.0e14, 2.0e15, 2000)
103
104     fig, ax = plt.subplots(figsize=(7, 5))
105
106     for label, work_function in WORK_FUNCTIONS.items():
107         V_stop = stopping_voltage(frequencies, work_function)
108
109         ax.plot(
110             frequencies,
111             V_stop,
112             label=f"{label} ( $\phi = {work_function:.2f}$  eV)"
113         )
114
115     ax.axhline(
116         0,
117         color="grey",
118         ls="--",
119         lw=1
120     )
121
122     ax.set_xlabel("Incident photon frequency / Hz")
123     ax.set_ylabel("Stopping voltage / V")
124     ax.set_title("Photoelectric effect: stopping voltage vs frequency")
125     ax.legend()
126
127     fig.tight_layout()
128     fig.savefig("task4_photoelectric_effect.png", dpi=150)
129
130     print("Saved task4_photoelectric_effect.png")
131
132     print("\nThreshold frequencies:")
133     for label, work_function in WORK_FUNCTIONS.items():
134         f0 = threshold_frequency(work_function)
135         print(f"{label}: {f0:.4e} Hz")

```

A.5 Task 4 Extension: Interactive Photoelectric Simulator (task4_extension.py)

```

1  """
2  BPhO Computational Physics Challenge 2026 - Quantum Mechanics
3  TASK #4 EXTENSION: Interactive photoelectric-effect simulation
4
5  Extension specified by the challenge:
6      "Create an animated app like the PhET demo."
7
8  This app provides an interactive visual model of the photoelectric effect.
9  The user can vary:
10
11      - the metal/work function,
12      - incident photon frequency,
13      - light intensity,
14      - stopping voltage.
15
16  The physics is based on Einstein's photoelectric equation:
17
18       $hf = \phi + K_{max}$ 
19
20  with
21
22       $K_{max} = e V_s$ 
23
24  so that
25
26       $V_s = hf/e - \phi/e$ .
27
28  For a work function expressed in eV,
29
30       $V_s = (h/e) f - \phi_{eV}$ .
31
32  The threshold frequency is
33
34       $f_0 = \phi/h$ .

```

```

35
36 Photons with  $f < f_0$  cannot eject electrons. Above threshold,
37 electrons are emitted with maximum kinetic energy
38
39  $K_{max} = hf - \phi$ .
40
41 The animation is deliberately a model rather than a microscopic
42 simulation: emitted electrons are represented by moving particles,
43 while their initial kinetic energies are sampled between zero and
44 the maximum allowed value. The number of emitted electrons is
45 controlled by the intensity parameter.
46
47 Reference for the work functions:
48 HyperPhysics, "Work Functions for Photoelectric Effect",
49 Georgia State University:
50 https://hyperphysics.phy-astr.gsu.edu/hbase/Tables/photoelec.html
51 """
52
53 import tkinter as tk
54 from tkinter import ttk
55 import random
56 import math
57
58 # -----
59 # Physical constants (SI)
60 # -----
61
62 H = 6.62607015e-34 # Planck constant, J s
63 E_CHARGE = 1.602176634e-19 # elementary charge, C
64
65 # Work functions in electron-volts.
66 # Reference: HyperPhysics / Handbook of Chemistry and Physics.
67 WORK_FUNCTIONS = {
68     "Cesium (Cs)": 2.10,
69     "Sodium (Na)": 2.28,
70     "Zinc (Zn)": 4.30,
71     "Copper (Cu)": 4.70,
72     "Platinum (Pt)": 6.35,
73 }
74
75
76 def threshold_frequency(work_function_eV: float) -> float:
77     """Return the photoelectric threshold frequency in Hz."""
78     return work_function_eV * E_CHARGE / H
79
80
81 def photon_energy_eV(frequency: float) -> float:
82     """Return photon energy  $hf$  in electron-volts."""
83     return H * frequency / E_CHARGE
84
85
86 def maximum_kinetic_energy_eV(
87     frequency: float,
88     work_function_eV: float
89 ) -> float:
90     """Return maximum photoelectron kinetic energy in eV."""
91     return max(
92         0.0,
93         photon_energy_eV(frequency) - work_function_eV
94     )
95
96
97 def stopping_voltage(
98     frequency: float,
99     work_function_eV: float
100 ) -> float:
101     """Return stopping voltage in volts."""
102     return maximum_kinetic_energy_eV(
103         frequency,
104         work_function_eV
105     )
106
107

```

```

108 class PhotoelectricApp:
109     """Animated Thinter demonstration of the photoelectric effect."""
110
111     def __init__(self, root: tk.Tk):
112         self.root = root
113         self.root.title("BPh0 2026 ---Photoelectric Effect")
114         self.root.geometry("1100x700")
115         self.root.minsize(900, 600)
116
117         self.running = True
118         self.electrons = []
119         self.photons = []
120
121         # User-controllable parameters.
122         self.metal = tk.StringVar(value="Cesium (Cs)")
123         self.frequency = tk.DoubleVar(value=8.0e14)
124         self.intensity = tk.DoubleVar(value=50.0)
125         self.stopping_voltage_value = tk.DoubleVar(value=0.0)
126
127         self._build_ui()
128         self._update_labels()
129         self._animate()
130
131         # -----
132         # UI construction
133         # -----
134
135     def _build_ui(self):
136         main = ttk.Frame(self.root, padding=12)
137         main.pack(fill="both", expand=True)
138
139         title = ttk.Label(
140             main,
141             text="Photoelectric Effect",
142             font=("Segoe UI", 20, "bold")
143         )
144         title.pack(anchor="w")
145
146         subtitle = ttk.Label(
147             main,
148             text="Einstein's photoelectric equation ---interactive model",
149             font=("Segoe UI", 10)
150         )
151         subtitle.pack(anchor="w", pady=(0, 10))
152
153         controls = ttk.Frame(main)
154         controls.pack(fill="x", pady=(0, 10))
155
156         # Metal selector.
157         ttk.Label(
158             controls,
159             text="Metal:"
160         ).grid(row=0, column=0, sticky="w", padx=(0, 8))
161
162         metal_box = ttk.Combobox(
163             controls,
164             textvariable=self.metal,
165             values=list(WORK_FUNCTIONS),
166             state="readonly",
167             width=18
168         )
169         metal_box.grid(row=0, column=1, sticky="w")
170         metal_box.bind(
171             "<<ComboboxSelected>>",
172             lambda event: self._update_labels()
173         )
174
175         # Frequency slider.
176         ttk.Label(
177             controls,
178             text="Frequency:"
179         ).grid(row=0, column=2, sticky="w", padx=(25, 8))
180

```

```

181     self.frequency_scale = tk.Scale(
182         controls,
183         variable=self.frequency,
184         from_=3.0e14,
185         to=2.0e15,
186         resolution=1.0e13,
187         orient="horizontal",
188         length=360,
189         showvalue=False,
190         command=lambda value: self._update_labels()
191     )
192     self.frequency_scale.grid(row=0, column=3, sticky="ew")
193
194     # Intensity slider.
195     ttk.Label(
196         controls,
197         text="Intensity:"
198     ).grid(row=1, column=0, sticky="w", pady=(8, 0))
199
200     self.intensity_scale = tk.Scale(
201         controls,
202         variable=self.intensity,
203         from_=0,
204         to=100,
205         resolution=1,
206         orient="horizontal",
207         length=250,
208         showvalue=False
209     )
210     self.intensity_scale.grid(
211         row=1,
212         column=1,
213         columnspan=2,
214         sticky="w",
215         pady=(8, 0)
216     )
217
218     # Stopping-voltage slider.
219     ttk.Label(
220         controls,
221         text="Stopping voltage:"
222     ).grid(row=1, column=3, sticky="w", pady=(8, 0))
223
224     self.stop_scale = tk.Scale(
225         controls,
226         variable=self.stopping_voltage_value,
227         from_=0,
228         to=10,
229         resolution=0.1,
230         orient="horizontal",
231         length=250,
232         showvalue=False,
233         command=lambda value: self._update_labels()
234     )
235     self.stop_scale.grid(
236         row=1,
237         column=4,
238         sticky="w",
239         pady=(8, 0)
240     )
241
242     controls.columnconfigure(3, weight=1)
243
244     # Main simulation canvas.
245     self.canvas = tk.Canvas(
246         main,
247         background="#081018",
248         highlightthickness=0
249     )
250     self.canvas.pack(
251         fill="both",
252         expand=True
253     )

```

```

254
255     self.canvas.bind(
256         "<Configure>",
257         lambda event: self._draw_static_scene()
258     )
259
260     # Information panel.
261     self.info = ttk.Label(
262         main,
263         text="",
264         font=("Consolas", 10)
265     )
266     self.info.pack(
267         anchor="w",
268         pady=(10, 0)
269     )
270
271     self.status = ttk.Label(
272         main,
273         text="",
274         font=("Segoe UI", 11, "bold")
275     )
276     self.status.pack(
277         anchor="w",
278         pady=(4, 0)
279     )
280
281     # -----
282     # Physics / state
283     # -----
284
285     def _current_parameters(self):
286         metal = self.metal.get()
287         phi = WORK_FUNCTIONS[metal]
288         frequency = self.frequency.get()
289
290         return (
291             metal,
292             phi,
293             frequency,
294             threshold_frequency(phi),
295             photon_energy_eV(frequency),
296             maximum_kinetic_energy_eV(frequency, phi),
297         )
298
299     def _update_labels(self):
300         (
301             metal,
302             phi,
303             frequency,
304             threshold,
305             photon_energy,
306             kinetic_energy
307         ) = self._current_parameters()
308
309         self.info.config(
310             text=(
311                 f"{metal} "
312                 f" $\phi$  = {phi:.2f} eV "
313                 f"f = {frequency / 1e14:.2f}  $\times 10^{14}$  Hz "
314                 f" $f_0$  = {threshold / 1e14:.2f}  $\times 10^{14}$  Hz "
315                 f" $hf$  = {photon_energy:.2f} eV "
316                 f" $K_{\max}$  = {kinetic_energy:.2f} eV"
317             )
318         )
319
320         if frequency < threshold:
321             self.status.config(
322                 text="NO PHOTOELECTRONS ---photon energy is below the work function"
323             )
324         else:
325             applied = self.stopping_voltage_value.get()
326

```

```

327         if applied >= kinetic_energy:
328             message = (
329                 "STOPPED ---applied stopping voltage is "
330                 "at least the required stopping potential"
331             )
332         else:
333             message = (
334                 f"PHOTOELECTRONS EMITTED ---stopping voltage required: "
335                 f"{kinetic_energy:.2f} V"
336             )
337
338         self.status.config(text=message)
339
340     # -----
341     # Drawing
342     # -----
343
344     def _draw_static_scene(self):
345         self.canvas.delete("static")
346
347         width = self.canvas.winfo_width()
348         height = self.canvas.winfo_height()
349
350         if width <= 1 or height <= 1:
351             return
352
353         surface_x = int(width * 0.58)
354         surface_top = int(height * 0.16)
355         surface_bottom = int(height * 0.88)
356
357         # Light source / photon side.
358         self.canvas.create_text(
359             90,
360             surface_top - 30,
361             text="PHOTONS",
362             fill="#d9edf7",
363             font=("Segoe UI", 10, "bold"),
364             tags="static"
365         )
366
367         # Metal plate.
368         self.canvas.create_rectangle(
369             surface_x,
370             surface_top,
371             surface_x + 35,
372             surface_bottom,
373             fill="#64707b",
374             outline="#b7c1c9",
375             width=2,
376             tags="static"
377         )
378
379         self.canvas.create_text(
380             surface_x + 18,
381             surface_bottom + 22,
382             text="metal",
383             fill="#d9edf7",
384             font=("Segoe UI", 10),
385             tags="static"
386         )
387
388         # Vacuum region.
389         self.canvas.create_text(
390             int(width * 0.78),
391             surface_top - 30,
392             text="VACUUM",
393             fill="#d9edf7",
394             font=("Segoe UI", 10, "bold"),
395             tags="static"
396         )
397
398         # Stopping-potential barrier.
399         barrier_x = int(width * 0.88)

```



```

400
401     self.canvas.create_line(
402         barrier_x,
403         surface_top,
404         barrier_x,
405         surface_bottom,
406         fill="#e4b44c",
407         width=3,
408         tags="static"
409     )
410
411     self.canvas.create_text(
412         barrier_x,
413         surface_bottom + 22,
414         text="collector",
415         fill="#d9edf7",
416         font=("Segoe UI", 10),
417         tags="static"
418     )
419
420     # Labels.
421     self.canvas.create_text(
422         surface_x - 80,
423         surface_bottom - 15,
424         text="photoemitting surface",
425         fill="#aebbc5",
426         font=("Segoe UI", 9),
427         tags="static"
428     )
429
430     def _draw_photon(self, x, y):
431         self.canvas.create_line(
432             x,
433             y,
434             x + 22,
435             y,
436             fill="#8fe3ff",
437             width=2,
438             tags="photon"
439         )
440         self.canvas.create_oval(
441             x + 18,
442             y - 4,
443             x + 26,
444             y + 4,
445             fill="#8fe3ff",
446             outline="",
447             tags="photon"
448         )
449
450     def _draw_electron(self, x, y):
451         self.canvas.create_oval(
452             x - 5,
453             y - 5,
454             x + 5,
455             y + 5,
456             fill="#f5d76e",
457             outline="",
458             tags="electron"
459         )
460
461     # -----
462     # Animation
463     # -----
464
465     def _spawn_photon(self):
466         width = self.canvas.winfo_width()
467         height = self.canvas.winfo_height()
468
469         if width <= 1 or height <= 1:
470             return
471
472         surface_x = int(width * 0.58)

```

```

473     surface_top = int(height * 0.16)
474     surface_bottom = int(height * 0.88)
475
476     y = random.randint(
477         surface_top + 15,
478         surface_bottom - 15
479     )
480
481     self.photons.append(
482         {
483             "x": 20.0,
484             "y": float(y),
485         }
486     )
487
488     def _process_photon_hits(self):
489         (
490             _metal,
491             phi,
492             frequency,
493             threshold,
494             _photon_energy,
495             kinetic_energy
496         ) = self._current_parameters()
497
498         width = self.canvas.wininfo_width()
499         surface_x = int(width * 0.58)
500
501         remaining = []
502
503         for photon in self.photons:
504             photon["x"] += 14
505
506             if photon["x"] >= surface_x:
507                 # Photon reaches the metal surface.
508                 if frequency >= threshold:
509                     # Sample an emitted electron energy up to K_max.
510                     # The animation is illustrative rather than a
511                     # microscopic scattering simulation.
512                     energy_fraction = random.uniform(0.15, 1.0)
513
514                     self.electrons.append(
515                         {
516                             "x": float(surface_x + 38),
517                             "y": float(photon["y"]),
518                             "speed": 2.0 + 5.0 * math.sqrt(
519                                 energy_fraction
520                             ),
521                         }
522                     )
523
524                     continue
525
526                 remaining.append(photon)
527
528         self.photons = remaining
529
530     def _update_electrons(self):
531         width = self.canvas.wininfo_width()
532         height = self.canvas.wininfo_height()
533
534         surface_x = int(width * 0.58)
535         collector_x = int(width * 0.88)
536
537         applied_voltage = self.stopping_voltage_value.get()
538         kinetic_max = maximum_kinetic_energy_eV(
539             self.frequency.get(),
540             WORK_FUNCTIONS[self.metal.get()]
541         )
542
543         remaining = []
544
545         for electron in self.electrons:

```

```

546         # A stopping voltage acts as an opposing barrier.
547         if applied_voltage >= kinetic_max and kinetic_max > 0:
548             electron["speed"] *= 0.94
549
550         if electron["speed"] < 0.12:
551             continue
552
553         electron["x"] += electron["speed"]
554
555         if electron["x"] >= collector_x:
556             continue
557
558         if electron["x"] < surface_x:
559             continue
560
561         remaining.append(electron)
562
563     self.electrons = remaining
564
565     def _animate(self):
566         self.canvas.delete("photon")
567         self.canvas.delete("electron")
568
569         # Spawn rate follows intensity.
570         if random.random() < self.intensity.get() / 120:
571             self._spawn_photon()
572
573         self._process_photon_hits()
574         self._update_electrons()
575
576         for photon in self.photons:
577             self._draw_photon(
578                 photon["x"],
579                 photon["y"]
580             )
581
582         for electron in self.electrons:
583             self._draw_electron(
584                 electron["x"],
585                 electron["y"]
586             )
587
588         self._update_labels()
589
590         self.root.after(
591             30,
592             self._animate
593         )
594
595     def main():
596         root = tk.Tk()
597
598         # Use a native ttk theme where available.
599         try:
600             ttk.Style(root).theme_use("clam")
601         except tk.TclError:
602             pass
603
604         PhotoelectricApp(root)
605         root.mainloop()
606
607     if __name__ == "__main__":
608         main()
609
610

```

A.6 Task 5: Bohr Model and Hydrogen Spectrum (task5.py)

```

1  """
2  BPhO Computational Physics Challenge 2026 - Quantum Mechanics
3  TASK #5: Hydrogen emission spectrum and Bohr's model

```

```

4
5 Create a graph of photon energy against wavelength for photon
6 emissions from hydrogen atoms due to transitions between electron
7 energy levels.
8
9 The Bohr model is derived here from two classical relations together
10 with quantisation of orbital angular momentum.
11
12 For an electron in a circular orbit of radius r around a proton,
13
14  $m_e v^2 / r = e^2 / (4 \pi \epsilon_0 r^2)$ 
15
16 and Bohr's angular-momentum quantisation condition is
17
18  $m_e v r = n \hbar$ ,
19
20 where  $n = 1, 2, 3, \dots$  and  $\hbar = h/(2 \pi)$ .
21
22 Combining these gives the allowed orbit radii
23
24  $r_n = 4 \pi \epsilon_0 \hbar^2 n^2 / (m_e e^2)$ 
25
26 and the corresponding electron energies
27
28  $E_n = -m_e e^4 / (8 \epsilon_0^2 \hbar^2 n^2)$ .
29
30 The ground-state energy is therefore calculated directly from the
31 physical constants rather than inserted as an unexplained 13.6 eV:
32
33  $E_1 = -m_e e^4 / (8 \epsilon_0^2 \hbar^2)$ 
34  $= -13.6057 \text{ eV}$ .
35
36 Thus,
37
38  $E_n = E_1 / n^2$ .
39
40 When an electron falls from an initial level  $n_i$  to a lower level  $n_f$ ,
41 the emitted photon has energy
42
43  $E_{\text{gamma}} = E_{n_i} - E_{n_f}$ 
44
45 and
46
47  $E_{\text{gamma}} = h c / \lambda$ .
48
49 This produces the hydrogen emission spectrum.
50
51 NOTE:
52 The numerical constants used here are standard SI values. The
53 calculated ground-state energy is 13.6057 eV, while the measured
54 ionisation energy of neutral hydrogen is approximately 13.5984 eV.
55 The small difference reflects the idealisations of the basic Bohr
56 model, including its treatment of the electron and proton masses.
57
58 References:
59 NIST CODATA, Fundamental Physical Constants:
60 https://physics.nist.gov/cuu/Constants/
61
62 NIST Atomic Spectra Database / Handbook of Basic Atomic Spectroscopic
63 Data, Hydrogen:
64 https://physics.nist.gov/PhysRefData/Handbook/Tables/hydrogentable1.htm
65 """
66
67 import os
68 from pathlib import Path
69
70 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
71 import matplotlib
72 matplotlib.use("Agg")
73 import numpy as np
74 import matplotlib.pyplot as plt
75
76

```

```

77 # Physical constants (SI)
78
79 H = 6.62607015e-34      # Planck constant, J s
80 C = 2.99792458e8       # speed of light, m/s
81 E_CHARGE = 1.602176634e-19 # elementary charge, C
82 M_E = 9.1093837139e-31  # electron mass, kg
83 EPSILON_0 = 8.8541878188e-12 # vacuum permittivity, F/m
84
85 HBAR = H / (2 * np.pi)
86
87
88 # -----
89 # Bohr model
90 # -----
91
92 def bohr_radius(n: int) -> float:
93     """
94     Return the Bohr-model orbital radius for level n in metres.
95
96     Starting from
97
98          $m_e v^2 / r = e^2 / (4 \pi \epsilon_0 r^2)$ 
99
100     and
101
102          $m_e v r = n \hbar$ ,
103
104     the allowed radius is
105
106          $r_n = 4 \pi \epsilon_0 \hbar^2 n^2 / (m_e e^2)$ .
107     """
108
109     return (
110         4 * np.pi * EPSILON_0 * HBAR**2 * n**2
111         / (M_E * E_CHARGE**2)
112     )
113
114
115 # Calculate the Bohr ground-state energy directly from the model.
116
117 # From the Coulomb force equation,
118
119 #  $m_e v^2 / r = e^2 / (4 \pi \epsilon_0 r^2)$ 
120
121 # we have
122
123 #  $m_e v^2 = e^2 / (4 \pi \epsilon_0 r)$ .
124
125 # Therefore the total electron energy is
126
127 #  $E = K + U$ 
128 #  $= 1/2 m_e v^2 - e^2 / (4 \pi \epsilon_0 r)$ 
129
130 # which becomes
131
132 #  $E = -e^2 / (8 \pi \epsilon_0 r)$ .
133
134 # Substituting the quantised radius gives
135
136 #  $E_n = -m_e e^4 / (8 \epsilon_0^2 \hbar^2 n^2)$ .
137
138 GROUND_STATE_ENERGY_J = (
139     -M_E * E_CHARGE**4
140     / (8 * EPSILON_0**2 * H**2)
141 )
142
143 # Convert joules to electron-volts.
144 GROUND_STATE_ENERGY_EV = GROUND_STATE_ENERGY_J / E_CHARGE
145
146
147 def bohr_energy(n: int) -> float:
148     """
149

```

```

150     Return the Bohr-model energy of hydrogen level n in eV.
151
152      $E_n = E_1 / n^2$ 
153     """
154
155     return GROUND_STATE_ENERGY_EV / n**2
156
157 # -----
158 # Photon emission
159 # -----
160
161 def photon_energy(n_initial: int, n_final: int) -> float:
162     """
163     Return emitted photon energy for a downward transition in eV.
164
165      $E_{\gamma} = E_{\text{initial}} - E_{\text{final}}$ 
166     """
167
168     if n_initial <= n_final:
169         raise ValueError("Emission requires n_initial > n_final.")
170
171     return (
172         bohr_energy(n_initial)
173         - bohr_energy(n_final)
174     )
175
176
177 def photon_wavelength(energy_eV: float) -> float:
178     """
179     Return photon wavelength in nm for a photon energy in eV.
180
181      $E_{\gamma} = h c / \lambda$ 
182     """
183
184     energy_J = energy_eV * E_CHARGE
185     wavelength_m = H * C / energy_J
186
187     return wavelength_m * 1e9
188
189
190 def plot_bohr_energy_levels() -> None:
191     """Plot Bohr energy levels and two representative emission transitions."""
192     fig, ax = plt.subplots(figsize=(7, 5.5))
193     # The first four levels are sufficiently separated to label clearly;
194     # higher levels visibly converge towards the ionisation limit.
195     levels = range(1, 5)
196
197     for n in levels:
198         energy = bohr_energy(n)
199         ax.hlines(energy, 0.35, 1.65, color="#2a6fbb", lw=2)
200         ax.text(1.72, energy, rf"$n={n}$ ({energy:.2f} eV)", va="center")
201
202     ax.text(0.38, -0.45, r"higher $n$ levels converge to $E=0$", color="#555555")
203
204     # Show a visible Balmer photon and an ultraviolet Lyman photon.
205     transitions = ((3, 2, "#d1495b", "656.5 nm"), (2, 1, "#6a4c93", "121.6 nm"))
206     for x, (n_i, n_f, colour, label) in zip((0.70, 1.30), transitions):
207         ax.annotate("", xy=(x, bohr_energy(n_f) + 0.10),
208                     xytext=(x, bohr_energy(n_i) - 0.10),
209                     arrowprops=dict(arrowstyle="->", color=colour, lw=2.5))
210         ax.text(x + 0.05, (bohr_energy(n_i) + bohr_energy(n_f)) / 2,
211                 rf"${n_i} \to {n_f}$ {label}", color=colour, va="center")
212
213     ax.axhline(0, color="black", lw=1, ls="--")
214     ax.text(1.72, 0, "ionisation limit", va="center")
215     ax.set(xlim=(0, 2.65), ylim=(-14.5, 1), xticks=[],
216           ylabel="Electron energy / eV",
217           title="Bohr model of hydrogen: quantised energy levels")
218     ax.spines[["top", "right", "bottom"]].set_visible(False)
219     fig.tight_layout()
220     fig.savefig("task5_bohr_model.png", dpi=180)
221
222

```

```

223
224 # -----
225 # Main
226 # -----
227
228 if __name__ == "__main__":
229
230     # --- Display the calculated fundamental Bohr quantities ---
231
232     print(
233         f"Calculated Bohr ground-state energy: "
234         f"{GROUND_STATE_ENERGY_EV:.7f} eV"
235     )
236
237     print(
238         f"Bohr radius: "
239         f"{bohr_radius(1) * 1e10:.6f} Angstrom"
240     )
241
242
243     # --- Calculate hydrogen emission transitions ---
244
245     # Include all downward transitions between n = 2 and n = 6.
246     #
247     # This produces the first several lines of the Lyman, Balmer,
248     # Paschen, Brackett and higher spectral series.
249
250     transitions = []
251
252     for n_initial in range(2, 7):
253
254         for n_final in range(1, n_initial):
255
256             energy = photon_energy(
257                 n_initial,
258                 n_final
259             )
260
261             wavelength = photon_wavelength(
262                 energy
263             )
264
265             transitions.append(
266                 (
267                     n_initial,
268                     n_final,
269                     energy,
270                     wavelength
271                 )
272             )
273
274
275     # Sort by wavelength for a clearer spectral plot.
276
277     transitions.sort(
278         key=lambda item: item[3]
279     )
280
281     wavelengths = np.array(
282         [item[3] for item in transitions]
283     )
284
285     energies = np.array(
286         [item[2] for item in transitions]
287     )
288
289
290     # --- Plot: Hydrogen emission spectrum ---
291
292     fig, ax = plt.subplots(
293         figsize=(8, 5)
294     )
295

```



```

296     ax.scatter(
297         wavelengths,
298         energies,
299         s=45
300     )
301
302
303     # Label only the Balmer transitions to keep the near-UV region readable.
304
305     for (
306         n_initial,
307         n_final,
308         energy,
309         wavelength
310     ) in transitions:
311
312         if n_final == 2 and n_initial in (3, 4, 5):
313             ax.annotate(
314                 rf"${n_initial}\rightarrow{n_final}$",
315                 (wavelength, energy),
316                 xytext=(5, 5),
317                 textcoords="offset points",
318                 fontsize=8
319             )
320
321
322     ax.set_xlabel(
323         "Photon wavelength / nm"
324     )
325
326     # The emission lines span ultraviolet through infrared wavelengths.
327     # A logarithmic wavelength axis makes all transition series visible.
328     ax.set_xscale("log")
329
330     ax.set_ylabel(
331         "Photon energy / eV"
332     )
333
334     ax.set_title(
335         "Hydrogen emission spectrum from the Bohr model"
336     )
337
338     fig.tight_layout()
339
340     fig.savefig(
341         "task5_hydrogen_emission_spectrum.png",
342         dpi=150
343     )
344     plot_bohr_energy_levels()
345
346
347     print(
348         "Saved task5_hydrogen_emission_spectrum.png and task5_bohr_model.png"
349     )
350
351
352     # --- Print transition data ---
353
354     print("\nHydrogen emission transitions:")
355     print(
356         "Transition   Wavelength / nm   "
357         "Photon energy / eV"
358     )
359
360     for (
361         n_initial,
362         n_final,
363         energy,
364         wavelength
365     ) in transitions:
366
367         print(
368             f"{n_initial} -> {n_final}"

```

```

369         f"           {wavelength:10.3f}"
370         f"           {energy:8.4f}"
371     )

```

A.7 Task 6: Electron Diffraction (task6.py)

```

1  """
2  BPhO Computational Physics Challenge 2026 - Quantum Mechanics
3  TASK #6: Electron diffraction from polycrystalline graphite
4
5  This model follows slides 40--42 of the supplied Challenge Presentation.
6
7  An electron accelerated through a potential difference V has kinetic energy
8
9       $e V = p^2 / (2 m_e),$ 
10
11  so its de Broglie wavelength is
12
13       $\lambda = h / \sqrt{2 m_e e V}.$ 
14
15  Constructive diffraction from graphite planes obeys Bragg's law
16
17       $n \lambda = 2 d \sin(\phi),$ 
18
19  where n is the diffraction order. A ring appears on the spherical phosphor
20  screen at
21
22       $x = r \sin(2 \phi), \quad r = 65 \text{ mm}.$ 
23
24  Combining the first two equations gives the presentation's Task 6a check:
25
26       $1 / \sqrt{V} = [2 \sqrt{2 m_e e} d / (n h)] \sin(\phi).$ 
27
28  Thus a plot of  $1/\sqrt{V}$  against  $\sin(\phi)$  is a straight line; the gradient
29  can recover the graphite layer spacing d. Both stated graphite spacings,
30  0.123 nm and 0.213 nm, are included.
31  """
32
33  from dataclasses import dataclass
34  import os
35  from pathlib import Path
36
37  # Make the script reliably runnable on machines without a desktop Python GUI.
38  os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
39  import matplotlib
40  matplotlib.use("Agg")
41  import matplotlib.pyplot as plt
42  import numpy as np
43
44
45  # SI constants (exact SI definitions where applicable)
46  H = 6.62607015e-34      # Planck constant / J s
47  E_CHARGE = 1.602176634e-19 # elementary charge / C
48  M_E = 9.1093837139e-31   # electron mass / kg
49  SCREEN_RADIUS_M = 65e-3
50
51
52  @dataclass(frozen=True)
53  class GraphitePlane:
54      """A graphite plane family used to form one series of diffraction rings."""
55
56      name: str
57      spacing_m: float
58      colour: str
59
60
61  PLANES = (
62      GraphitePlane("d1 = 0.123 nm", 0.123e-9, "#2a6fbb"),
63      GraphitePlane("d2 = 0.213 nm", 0.213e-9, "#d1495b"),
64  )
65

```

```

66
67 def electron_wavelength(voltage_V: float) -> float:
68     """Return the non-relativistic de Broglie wavelength in metres."""
69     if voltage_V <= 0:
70         raise ValueError("Accelerating voltage must be positive.")
71     return h / np.sqrt(2 * m_e * e_charge * voltage_V)
72
73
74 def diffraction_rings(voltage_V: float, plane: GraphitePlane):
75     """Return all physically allowed (order, phi, radius) rings for one plane.
76
77     Bragg's law requires n*lambda/(2*d) <= 1. The order is therefore not
78     arbitrarily truncated: every allowed integer order is returned.
79     """
80     wavelength = electron_wavelength(voltage_V)
81     maximum_order = int(np.floor(2 * plane.spacing_m / wavelength))
82     rings = []
83
84     for order in range(1, maximum_order + 1):
85         sin_phi = order * wavelength / (2 * plane.spacing_m)
86         phi = np.arcsin(sin_phi)
87         radius_m = SCREEN_RADIUS_M * np.sin(2 * phi)
88         rings.append((order, phi, radius_m))
89
90     return rings
91
92
93 def recovered_spacing(gradient: float, order: int = 1) -> float:
94     """Recover d from the Task 6a gradient in SI units."""
95     return gradient * order * h / (2 * np.sqrt(2 * m_e * e_charge))
96
97
98 def draw_screen(ax, voltage_kV: float) -> None:
99     """Draw a face-on spherical screen and every allowed ring at one voltage."""
100     screen = plt.Circle((0, 0), SCREEN_RADIUS_M * 1e3,
101                          facecolor="#e9fff0", edgecolor="#255c3b", lw=2)
102     ax.add_patch(screen)
103     ax.plot(0, 0, "+", color="#333333", ms=8, mew=1.5)
104
105     for plane in PLANES:
106         for order, _, radius_m in diffraction_rings(voltage_kV * 1e3, plane):
107             ring = plt.Circle((0, 0), radius_m * 1e3, fill=False,
108                               color=plane.colour, lw=1.15, alpha=0.9)
109             ax.add_patch(ring)
110
111     radius_mm = SCREEN_RADIUS_M * 1e3
112     ax.set(xlim=(-radius_mm, radius_mm), ylim=(-radius_mm, radius_mm),
113           aspect="equal", title=f"Model phosphor screen: {voltage_kV:g} kV")
114     ax.set_xticks([])
115     ax.set_yticks([])
116
117
118 def main() -> None:
119     """Create screen-ring models and the required straight-line validation."""
120     fig, axes = plt.subplot_mosaic(
121         [["one", "three"], ["five", "validation"]], figsize=(12, 10)
122     )
123
124     for axis_name, voltage_kV in (("one", 1), ("three", 3), ("five", 5)):
125         draw_screen(axes[axis_name], voltage_kV)
126
127     # Legend placed beneath the screen series, without obscuring rings.
128     handles = [plt.Line2D([], [], color=p.colour, lw=2, label=p.name)
129               for p in PLANES]
130     axes["five"].legend(handles=handles, loc="upper center",
131                        bbox_to_anchor=(0.5, -0.05), frameon=False)
132
133     # Task 6a: use first-order rings at a range of voltages. The result is
134     # linear for each spacing; least-squares gradients independently recover d.
135     validation_ax = axes["validation"]
136     voltages = np.linspace(1e3, 5e3, 15)
137     for plane in PLANES:
138         phi = np.array([diffraction_rings(V, plane)[0][1] for V in voltages])

```

```

139     x = np.sin(phi)
140     y = 1 / np.sqrt(voltages)
141     gradient, intercept = np.polyfit(x, y, 1)
142     fitted_spacing_nm = recovered_spacing(gradient) * 1e9
143
144     validation_ax.scatter(x, y, color=plane.colour, s=26, zorder=3)
145     validation_ax.plot(
146         x, gradient * x + intercept, color=plane.colour, lw=1.5,
147         label=(f"{plane.name}; fit d = {fitted_spacing_nm:.3f} nm")
148     )
149
150     validation_ax.set(
151         xlabel=r"$\sin\phi$",
152         ylabel=r"$1 / \sqrt{V}$ / $V^{-1/2}$",
153         title=r"Task 6a check: $1/\sqrt{V}$ vs $\sin\phi$",
154     )
155     validation_ax.grid(alpha=0.25)
156     validation_ax.legend(fontsize=9)
157
158     fig.suptitle("Electron diffraction by graphite: 1--5 kV", fontsize=16, y=0.98)
159     fig.tight_layout()
160     output = Path(__file__).with_name("task6_electron_diffraction.png")
161     fig.savefig(output, dpi=180)
162     print(f"Saved {output.name}")
163
164     for voltage_kV in (1, 3, 5):
165         counts = [len(diffraction_rings(voltage_kV * 1e3, plane)) for plane in PLANES]
166         print(f"{voltage_kV} kV: {counts[0]} rings for d1; {counts[1]} rings for d2")
167
168
169 if __name__ == "__main__":
170     main()

```

A.8 Task 7: Particle in a Box (task7.py)

```

1  """BPhO 2026 Task 7: normalised particle-in-a-box eigenstates.
2
3  For a box of width a,  $\psi_n(x) = \sqrt{2/a} \sin(n \pi x/a)$  and
4   $E_n = n^2 \pi^2 \hbar^2 / (2 m a^2)$ . The plot uses an electron in a 1 nm box.
5  """
6
7  import os
8  from pathlib import Path
9
10 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
11 import matplotlib
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14 import numpy as np
15
16 HBAR = 1.054571817e-34
17 M_E = 9.1093837139e-31
18 E_CHARGE = 1.602176634e-19
19
20
21 def energy(n: int, mass: float, width_m: float) -> float:
22     """Particle-in-a-box energy in joules."""
23     return n**2 * np.pi**2 * HBAR**2 / (2 * mass * width_m**2)
24
25
26 def probability_density(x_m: np.ndarray, n: int, width_m: float) -> np.ndarray:
27     """Return  $|\psi_n(x)|^2$ , zero outside the box."""
28     density = np.zeros_like(x_m)
29     inside = (x_m > 0) & (x_m < width_m)
30     density[inside] = (2 / width_m) * np.sin(n * np.pi * x_m[inside] / width_m)**2
31     return density
32
33
34 def main() -> None:
35     width_m = 1e-9
36     x_m = np.linspace(0, width_m, 1200)

```

```

37 ns = np.arange(1, 7)
38
39 fig, (ax_energy, ax_density) = plt.subplots(1, 2, figsize=(12, 5))
40 energies_eV = np.array([energy(n, M_E, width_m) / E_CHARGE for n in ns])
41 ax_energy.scatter(ns, energies_eV, s=60, color="#2a6fbb", zorder=3)
42 ax_energy.vlines(ns, 0, energies_eV, color="#2a6fbb", alpha=0.35)
43 ax_energy.set(xlabel="Quantum number, n", ylabel="Energy / eV",
44              title=r"Particle in a 1 nm box:  $E_n \propto n^2$ ")
45 ax_energy.set_xticks(ns)
46 ax_energy.grid(alpha=0.25)
47
48 for n in range(1, 5):
49     ax_density.plot(x_m * 1e9, probability_density(x_m, n, width_m) * 1e-9,
50                  label=r"$n={n}$")
51 ax_density.set(xlabel="Position in box, x / nm", ylabel=r"Probability density,  $|\psi_n|^2 / \text{nm}^{-1}$ ",
52              title="Normalised stationary-state probability densities")
53 ax_density.legend()
54 ax_density.grid(alpha=0.25)
55 fig.tight_layout()
56 fig.savefig("task7_particle_in_a_box.png", dpi=180)
57 print("Saved task7_particle_in_a_box.png")
58
59
60 if __name__ == "__main__":
61     main()

```

A.9 Task 7 Extension: Uncertainty Principle (task7_extension.py)

```

1 """Task 7 extension: verify Heisenberg uncertainty for a particle in a box.
2
3 For  $\psi_n(x) = \sqrt{2/a} \sin(n\pi x/a)$ , Q2 of the supplied QM3 sheet gives
4  $\Delta x \Delta p = (n\pi/2) \sqrt{1/3 - 2/(\pi^2 n^2)}$ . This program evaluates it
5 alongside  $\hbar/2$ , both analytically and from numerical integrals.
6 """
7
8 import os
9 from pathlib import Path
10
11 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
12 import matplotlib
13 matplotlib.use("Agg")
14 import matplotlib.pyplot as plt
15 import numpy as np
16
17 HBAR = 1.054571817e-34
18
19
20 def analytic_uncertainty_product(n: np.ndarray) -> np.ndarray:
21     """Return  $\Delta x$  times  $\Delta p$  in J s for the nth stationary state."""
22     return HBAR * n * np.pi / 2 * np.sqrt(1 / 3 - 2 / (np.pi**2 * n**2))
23
24
25 def numerical_uncertainty_product(n: int, width_m: float) -> float:
26     """Integrate  $|\psi|^2$  to calculate  $\Delta x$  and use  $\langle p^2 \rangle = (n\pi\hbar/a)^2$ ."""
27     x = np.linspace(0, width_m, 100_001)
28     density = 2 / width_m * np.sin(n * np.pi * x / width_m)**2
29     mean_x = np.trapezoid(x * density, x)
30     mean_x2 = np.trapezoid(x**2 * density, x)
31     delta_x = np.sqrt(mean_x2 - mean_x**2)
32     delta_p = n * np.pi * HBAR / width_m #  $\langle p \rangle = 0$  in an energy eigenstate
33     return delta_x * delta_p
34
35
36 def main() -> None:
37     n = np.arange(1, 13)
38     analytical = analytic_uncertainty_product(n)
39     numerical = np.array([numerical_uncertainty_product(int(level), 1e-9) for level in n])
40
41     fig, ax = plt.subplots(figsize=(8, 5))
42     ax.plot(n, analytical / HBAR, "o-", label="Analytic result", color="#2a6fbb")

```

```

43 ax.plot(n, numerical / HBAR, "x", ms=8, mew=2, label="Numerical integral", color="#d1495b")
44 ax.axhline(0.5, color="black", ls="--", label=r"Heisenberg limit, $\hbar/2$")
45 ax.set(xlabel="Quantum number, n", ylabel=r"$\Delta x$, $\Delta p / \hbar$",
46       title="Particle in a box obeys the uncertainty principle")
47 ax.set_xticks(n)
48 ax.grid(alpha=0.25)
49 ax.legend()
50 fig.tight_layout()
51 fig.savefig("task7_uncertainty_principle.png", dpi=180)
52 print(f"n=1: Delta-x Delta-p = {analytical[0] / HBAR:.6f} hbar")
53 print("Saved task7_uncertainty_principle.png")
54
55
56 if __name__ == "__main__":
57     main()

```

A.10 Task 8: Photon Polarisation / Quantum Cryptography (task8.py)

```

1  """BPhO 2026 Task 8: classical and quantum photon-polarisation mismatch.
2
3  The formulae are those given on presentation slides 55--58. Angles theta and
4 phi specify Alice's and Bob's detector bases. Edit THETA_DEG and PHI_DEG to
5 use this file as a visual calculator.
6  """
7
8  import os
9  from pathlib import Path
10
11  os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
12  import matplotlib
13  matplotlib.use("Agg")
14  import matplotlib.pyplot as plt
15  import numpy as np
16
17  THETA_DEG = -30.0
18  PHI_DEG = 30.0
19
20
21  def classical_mismatch(theta: np.ndarray, phi: np.ndarray) -> np.ndarray:
22      return 1 - np.cos(theta)**2 * np.cos(phi)**2 - np.sin(theta)**2 * np.sin(phi)**2
23
24
25  def quantum_mismatch(theta: np.ndarray, phi: np.ndarray) -> np.ndarray:
26      return np.sin(phi - theta)**2
27
28
29  def main() -> None:
30      theta = np.deg2rad(THETA_DEG)
31      phi = np.deg2rad(PHI_DEG)
32      angle_deg = np.linspace(-90, 90, 500)
33      angle = np.deg2rad(angle_deg)
34      difference = PHI_DEG - THETA_DEG
35
36      fig, (ax_curve, ax_bar) = plt.subplots(1, 2, figsize=(12, 5))
37      ax_curve.plot(angle_deg, classical_mismatch(theta, angle), label="Classical", lw=2)
38      ax_curve.plot(angle_deg, quantum_mismatch(theta, angle), label="Quantum", lw=2)
39      ax_curve.axvline(PHI_DEG, color="grey", ls="--", lw=1)
40      ax_curve.plot(PHI_DEG, classical_mismatch(theta, phi), "o", color="#2a6fbb")
41      ax_curve.plot(PHI_DEG, quantum_mismatch(theta, phi), "o", color="#d1495b")
42      ax_curve.set(xlabel=r"Bob's angle, $\phi$ / degrees", ylabel="Mismatch probability",
43                 title=rf"Alice fixed at $\theta$={THETA_DEG:g}^\circ$")
44      ax_curve.set_ylim(0, 1)
45      ax_curve.grid(alpha=0.25)
46      ax_curve.legend()
47
48      values = [classical_mismatch(theta, phi), quantum_mismatch(theta, phi)]
49      bars = ax_bar.bar(["Classical", "Quantum"], values, color=["#2a6fbb", "#d1495b"])
50      for bar, value in zip(bars, values):
51          ax_bar.text(bar.get_x() + bar.get_width() / 2, value + 0.025, f"{value:.3f}", ha="center")
52      ax_bar.set(ylim=(0, 1), ylabel="Mismatch probability",

```

```

53         title=rf"Calculator: $\theta$={THETA_DEG:g}^\circ$, $\phi$={PHI_DEG:g}^\circ$ ($\phi$-$\theta$={
54             difference:g}^\circ$)")
55     ax_bar.grid(axis="y", alpha=0.25)
56     fig.tight_layout(w_pad=3.0)
57     fig.savefig("task8_quantum_cryptography.png", dpi=180)
58     print("Saved task8_quantum_cryptography.png")
59
60 if __name__ == "__main__":
61     main()

```

A.11 Task 9: Compton Scattering (task9.py)

```

1  """BPhO 2026 Task 9: relativistic Compton scattering of 100 keV photons.
2
3  The presentation gives Delta lambda = h/(m_e c) (1-cos(theta)). Momentum and
4  energy conservation give the recoil-electron momentum, speed and angle.
5  """
6
7  import os
8  from pathlib import Path
9
10 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
11 import matplotlib
12 matplotlib.use("Agg")
13 import matplotlib.pyplot as plt
14 import numpy as np
15
16 H = 6.62607015e-34
17 C = 2.99792458e8
18 M_E = 9.1093837139e-31
19 E_CHARGE = 1.602176634e-19
20 PHOTON_ENERGY_KEV = 100.0
21
22
23 def compton_results(theta: np.ndarray, photon_energy_keV: float = PHOTON_ENERGY_KEV):
24     """Return fractional shift, electron speed and recoil angle for theta."""
25     initial_energy = photon_energy_keV * 1e3 * E_CHARGE
26     wavelength = H * C / initial_energy
27     compton_wavelength = H / (M_E * C)
28     delta_wavelength = compton_wavelength * (1 - np.cos(theta))
29     scattered_wavelength = wavelength + delta_wavelength
30     scattered_energy = H * C / scattered_wavelength
31
32     initial_momentum = initial_energy / C
33     scattered_momentum = scattered_energy / C
34     p_x = initial_momentum - scattered_momentum * np.cos(theta)
35     p_y = scattered_momentum * np.sin(theta)
36     recoil_angle = np.arctan2(p_y, p_x)
37
38     kinetic_energy = initial_energy - scattered_energy
39     gamma = 1 + kinetic_energy / (M_E * C**2)
40     speed = C * np.sqrt(1 - gamma**-2)
41     return delta_wavelength / wavelength, speed, recoil_angle
42
43
44 def main() -> None:
45     theta = np.linspace(0, np.pi, 1000)
46     fractional_shift, speed, recoil_angle = compton_results(theta)
47     theta_deg = np.rad2deg(theta)
48
49     fig, axes = plt.subplots(1, 3, figsize=(14, 4.5))
50     axes[0].plot(theta_deg, fractional_shift, color="#2a6fbb")
51     axes[0].set(ylabel=r"$\Delta\lambda/\lambda$", title="Fractional wavelength shift")
52     axes[1].plot(theta_deg, speed / C, color="#d1495b")
53     axes[1].set(ylabel=r"Electron recoil speed, $v/c$", title="Electron recoil speed")
54     axes[2].plot(theta_deg, np.rad2deg(recoil_angle), color="#3a9d5d")
55     axes[2].set(ylabel=r"Electron recoil angle, $\phi$ / degrees", title="Electron recoil angle")
56     for ax in axes:
57         ax.set(xlabel=r"Photon scattering angle, $\theta$ / degrees", xlim=(0, 180))
58     ax.grid(alpha=0.25)

```

```

59     fig.suptitle(f"Compton scattering of a {PHOTON_ENERGY_KEV:g} keV photon", fontsize=15)
60     fig.tight_layout()
61     fig.savefig("task9_compton_scattering.png", dpi=180)
62     print("Saved task9_compton_scattering.png")
63
64
65 if __name__ == "__main__":
66     main()

```

A.12 Task 10: Hydrogenic Orbitals (task10.py)

```

1  """BPhO 2026 Task 10: 2D and 3D hydrogenic-orbital probability densities.
2
3  The code evaluates normalised radial functions and spherical harmonics without
4  external scientific libraries. Change ORBITALS to select valid (n, l, m)
5  quantum numbers. The 3D plots retain points above 15% of each density maximum,
6  matching the presentation's suggested visualisation threshold.
7  """
8
9  import math
10 import os
11 from pathlib import Path
12
13 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).with_name(".matplotlib")))
14 import matplotlib
15 matplotlib.use("Agg")
16 import matplotlib.pyplot as plt
17 import numpy as np
18
19 A0 = 5.29177210903e-11 # Bohr radius / m
20 ORBITALS = ((1, 0, 0, "1s"), (2, 1, 0, "2pz"),
21             (3, 2, 0, "3dz2"), (4, 3, 2, "4f, m=2"))
22
23
24 def associated_laguerre(order: int, alpha: int, x: np.ndarray) -> np.ndarray:
25     """Generalised Laguerre polynomial  $L_{\text{order}}^{\alpha}(x)$ , by recurrence."""
26     if order == 0:
27         return np.ones_like(x)
28     if order == 1:
29         return 1 + alpha - x
30     previous = np.ones_like(x)
31     current = 1 + alpha - x
32     for k in range(2, order + 1):
33         next_value = ((2 * k - 1 + alpha - x) * current - (k - 1 + alpha) * previous) / k
34         previous, current = current, next_value
35     return current
36
37
38 def associated_legendre(l: int, m: int, x: np.ndarray) -> np.ndarray:
39     """Associated Legendre  $P_l^m(x)$  for non-negative m."""
40     m = abs(m)
41     p_mm = (-1)**m * math.prod(range(1, 2 * m, 2)) * (1 - x**2)**(m / 2)
42     if l == m:
43         return p_mm
44     p_m1m = x * (2 * m + 1) * p_mm
45     if l == m + 1:
46         return p_m1m
47     previous, current = p_mm, p_m1m
48     for degree in range(m + 2, l + 1):
49         next_value = ((2 * degree - 1) * x * current - (degree + m - 1) * previous) / (degree - m)
50         previous, current = current, next_value
51     return current
52
53
54 def probability_density(x: np.ndarray, y: np.ndarray, z: np.ndarray, n: int, l: int, m: int) -> np.ndarray:
55     """Return  $|\psi_{nlm}|^2$  for hydrogen ( $Z=1$ ), in SI volume-density units."""
56     if not (n >= 1 and 0 <= l < n and -l <= m <= l):
57         raise ValueError("Require n >= 1, 0 <= l < n and -l <= m <= l.")
58     r = np.sqrt(x**2 + y**2 + z**2)
59     rho = 2 * r / (n * A0)
60     radial_norm = np.sqrt((2 / (n * A0))**3 * math.factorial(n - l - 1) /

```



```

61         (2 * n * math.factorial(n + 1)))
62     radial = radial_norm * np.exp(-rho / 2) * rho**l * associated_laguerre(n - l - 1, 2 * l + 1, rho)
63     cos_theta = np.divide(z, r, out=np.ones_like(r), where=r > 0)
64     abs_m = abs(m)
65     angular = ((2 * l + 1) / (4 * np.pi) * math.factorial(l - abs_m) /
66               math.factorial(l + abs_m) * associated_legendre(l, abs_m, cos_theta)**2)
67     return radial**2 * angular
68
69
70 def main() -> None:
71     extent_a0 = 22
72     coordinate = np.linspace(-extent_a0, extent_a0, 260) * A0
73     # Use the y=0 x-z plane. Unlike the z=0 plane, it also displays orbitals
74     # whose angular factor vanishes in the equatorial plane.
75     x, z = np.meshgrid(coordinate, coordinate)
76     y = np.zeros_like(x)
77     fig, axes = plt.subplots(2, 2, figsize=(10, 9), constrained_layout=True)
78     for ax, (n, l, m, label) in zip(axes.flat, ORBITALS):
79         density = probability_density(x, y, z, n, l, m)
80         image = ax.imshow(density / density.max(), extent=(-extent_a0, extent_a0, -extent_a0, extent_a0),
81                          origin="lower", cmap="magma", vmin=0, vmax=1)
82         ax.set(title=f"{label}: $n={n}$, $l={l}$, $m={m}$", xlabel=r"$x / $a_0$", ylabel=r"$z / $a_0$", aspect="
83               equal")
84     fig.colorbar(image, ax=axes, label=r"$|\psi|^2 / |\psi|^2_{\max}$", shrink=0.8)
85     fig.savefig("task10_hydrogenic_orbitals_2d.png", dpi=180)
86
87     # 3D coloured-glass analogue: dense points above 15% of the maximum.
88     grid = np.linspace(-14, 14, 45) * A0
89     x3, y3, z3 = np.meshgrid(grid, grid, grid, indexing="ij")
90     fig3 = plt.figure(figsize=(11, 5))
91     for index, (n, l, m, label) in enumerate(ORBITALS[1:3], start=1):
92         ax = fig3.add_subplot(1, 2, index, projection="3d")
93         density = probability_density(x3, y3, z3, n, l, m)
94         mask = density > 0.15 * density.max()
95         scaled = density[mask] / density.max()
96         ax.scatter(x3[mask] / A0, y3[mask] / A0, z3[mask] / A0, c=scaled,
97                  cmap="plasma", s=5, alpha=0.65, linewidths=0)
98         ax.set(title=f"3D probability density: {label}", xlabel=r"$x / $a_0$", ylabel=r"$y / $a_0$", zlabel=r"$z / $a_0$")
99         ax.set_box_aspect((1, 1, 1))
100         ax.view_init(elev=20, azim=-55)
101     fig3.tight_layout()
102     fig3.savefig("task10_hydrogenic_orbitals_3d.png", dpi=180)
103     print("Saved task10_hydrogenic_orbitals_2d.png and task10_hydrogenic_orbitals_3d.png")
104
105 if __name__ == "__main__":
106     main()

```